LinuC レベル1 試験対策教科書 — Version 10.0 対応

- 対象読者: Linux をこれから学ぶ方、および実務1年目までのエンジニア
- レベル: 初級〜中級
- 想定学習時間: 40〜60時間（1章あたり2〜4時間＋練習問題）
- 基準: LinuC-1 Version 10.0（2026年時点の公式出題範囲に準拠）
- 試験: 101試験／102試験（両方の合格で LinuC レベル1 認定）

はじめに

LinuC レベル1は、Linux システムの構築・運用・管理に必要な「基本操作」と「基礎知識」を問う認定試験です。難解な理論を暗記する試験ではなく、**実際のコマンド操作と設定ファイルの意味を理解しているか**が問われます。本書は、101試験と102試験の公式出題範囲（Version 10.0）を1冊にまとめ、試験直前の確認にも使える構成にしました。

本書の特徴は次の3点です。

1. **公式出題範囲の主題番号（1.01〜1.11）に沿った構成**。試験範囲の抜け漏れを防ぎます。
2. **コマンド例と設定ファイル例を豊富に掲載**。暗記ではなく「なぜそうなるか」を理解できます。
3. **各章末の練習問題と巻末の解答・解説**。知識の定着と本番形式の確認ができます。

> 試験制度・出題範囲は改訂されることがあります。受験前には必ず LinuC 公式サイト の最新情報を確認してください。本書は2026年時点の Version 10.0 を基準にしています。

学習目標

本書を読み終えたとき、次のことができる状態を目指します。

- Linux のインストール、起動・停止、仮想マシン・コンテナの基本操作ができる。
- ファイル・ディレクトリの操作、パーミッション、リンク、検索を理解して実行できる。
- パイプ・リダイレクト・正規表現・テキスト処理を使いこなせる。
- apt／yum と dpkg／rpm の違いを理解し、パッケージを管理できる。
- ディスク、パーティション、ファイルシステム、LVM の基本を理解している。
- Bash の環境設定と、条件分岐・ループ・引数処理を含むスクリプトを書ける。
- TCP/IP の基礎と、Linux のネットワーク設定・トラブルシューティングができる。
- ユーザー管理、cron／at、ログ、時刻同期、メール配送の基本を理解している。
- セキュリティ管理、SSH、GPG、クラウドセキュリティの基礎を理解している。
- オープンソースの定義と主要ライセンスの違いを説明できる。

LinuC とは

LinuC（リナック）は、LPI-Japan が実施する Linux 技術者認定試験です。特定のディストリビューションやベンダーに依存せず、Linux の共通技術を問う点が特徴です。レベル1は「Linux システムの構築・運用・管理の専門家」の入口に位置づけられ、すべての IT 技術者の基礎となる知識を証明します。

レベル	試験	対象
レベル1	101、102	Linux の基本操作・システム管理
レベル2	201、202	ネットワーク・サーバー構築を含む応用管理
レベル3	300番台	大規模・高信頼システムの設計運用
レベル4	システムアーキテクト	システム全体のアーキテクチャ設計

レベル1の認定には、**101試験と102試験の両方に合格**する必要があります。受験順は自由ですが、2試験は5年以内に合格しなければなりません。認定後も「有意性の期限」があり、認定日から5年以内の再認定または上位レベル取得が求められます。

試験概要（2026年時点）

項目	内容
正式名称	101試験: LinuC Level1 Exam 101／102試験: LinuC Level1 Exam 102
出題範囲	LinuC-1 Version 10.0
試験方式	CBT（コンピュータベーストテスト）。選択式が中心で、キーボード入力問題も一部あり
問題数	1試験あたり約60問
試験時間	90分（冒頭のアンケート等を除くと実質約85分）
受験費用	1試験あたり16,500円（税込）
合格基準	非公開。各分野をバランスよく得点することが重要
認定要件	101試験と102試験の両方に合格（5年以内）
学習期間の目安	1〜3か月

CBT では、後から見直したい問題にフラグを付けられます。1問あたり約1分30秒が目安です。分からない問題で止まらず、**1周目で解ける問題を確実に取り、2周目で迷った問題を見直す**のが定石です。

101試験と102試験の違い

試験	主な内容	公式主題
101	インストール、ファイル操作、コマンド、パッケージ、ディスク、ハードウェア	1.01〜1.05
102	シェルスクリプト、ネットワーク、システム管理、サービス、セキュリティ、OSS文化	1.06〜1.11

101は「Linux の基本的な使い方と管理」、102は「ネットワーク・サービス・セキュリティを含む運用」と捉えると整理しやすくなります。

本書の使い方

1. **まず各章の「学習目標」を読む**。何を覚えるべきかが分かります。
2. **本文を読み、コマンド例を実際に手元で試す**。LinuC は暗記より体験が大切です。
3. **章末の練習問題を解く**。間違えた問題は本文に戻って確認します。
4. **巻末の「コマンド早見表」で最終確認**。試験前日はここを繰り返し見るのがおすすめです。

学習環境は、VirtualBox／VMware／WSL2／クラウドのいずれでも構いません。Ubuntu 系と Rocky Linux（または AlmaLinux）系の両方に触れておくと、apt と yum、Debian 系と Red Hat 系の違いが理解しやすくなります。

学習プランの例

期間	進め方
2週間（集中）	1日2章＋練習問題。土日で総復習と模擬問題
1か月（標準）	平日1章、週末に復習。101→102の順に進める
3か月（じっくり）	週2章。実際に仮想マシンを構築し、サーバー設定を練習

目次

- はじめに
- 学習目標
- LinuC とは
- 試験概要（2026年時点）
    - 101試験と102試験の違い
- 本書の使い方
    - 学習プランの例
- 目次
- 第1章 Linuxのインストールと仮想マシン・コンテナの利用
    - この章で学ぶこと
    - 1.1 Linux のインストール
    - 1.2 起動・停止と SSH 接続
    - 1.3 仮想マシンとコンテナ
    - 1.4 ブートプロセスと systemd
    - 1.5 プロセスの生成・監視・終了
    - 1.6 デスクトップ環境（X Window System）
    - 本章のまとめ
    - 練習問題

- 第2章 ファイル・ディレクトリの操作と管理
  - この章で学ぶこと
  - 2.1 パーミッションと所有者
  - 2.2 基本的なファイル管理
  - 2.3 ハードリンクとシンボリックリンク
  - 2.4 FHS とファイル検索
  - 本章のまとめ
  - 練習問題
- 第3章 GNUとUnixのコマンド
  - この章で学ぶこと
  - 3.1 コマンドラインの操作
  - 3.2 フィルタによるテキスト処理
  - 3.3 ストリーム・パイプ・リダイレクト
  - 3.4 正規表現と grep／sed
  - 3.5 vi エディタ
  - 本章のまとめ
  - 練習問題
- 第4章 リポジトリとパッケージ管理
  - この章で学ぶこと
  - 4.1 パッケージ管理の基礎
  - 4.2 apt によるパッケージ管理
  - 4.3 dpkg による Debian パッケージ管理
  - 4.4 yum によるパッケージ管理
  - 4.5 rpm による RPM パッケージ管理
  - 4.6 どちらを使うべきか
  - 本章のまとめ
  - 練習問題
- 第5章 ハードウェア、ディスク、パーティション、ファイルシステム
  - この章で学ぶこと
  - 5.1 ハードウェアの基礎
  - 5.2 パーティションとディスクレイアウト
  - 5.3 LVM
  - 5.4 ファイルシステムの作成とマウント
  - 本章のまとめ
  - 練習問題
- 第6章 シェルおよびスクリプト
  - この章で学ぶこと
  - 6.1 シェル環境のカスタマイズ
  - 6.2 Bash スクリプトの基本
  - 本章のまとめ
  - 練習問題

- 第7章 ネットワークの基礎
  - この章で学ぶこと
  - 7.1 インターネットプロトコルの基礎
  - 7.2 ネットワーク設定
  - 7.3 ネットワークの問題解決
  - 7.4 クライアント側の DNS 設定
  - 本章のまとめ
  - 練習問題
- 第8章 システム管理
  - この章で学ぶこと
  - 8.1 アカウント管理
  - 8.2 ジョブスケジューリング
  - 8.3 ローカライゼーションと国際化
  - 本章のまとめ
  - 練習問題
- 第9章 重要なシステムサービス
  - この章で学ぶこと
  - 9.1 システム時刻の管理
  - 9.2 システムのログ
  - 9.3 メール配送エージェント（MTA）
  - 本章のまとめ
  - 練習問題
- 第10章 セキュリティ
  - この章で学ぶこと
  - 10.1 セキュリティ管理業務
  - 10.2 ホストのセキュリティ設定
  - 10.3 暗号化によるデータの保護
  - 10.4 クラウドセキュリティの基礎
  - 本章のまとめ
  - 練習問題
- 第11章 オープンソースの文化
  - この章で学ぶこと
  - 11.1 オープンソースの概念
  - 11.2 主要なライセンス
  - 11.3 コミュニティとエコシステム
  - 本章のまとめ
  - 練習問題
- 第12章 総まとめと合格戦略
  - この章で学ぶこと
  - 12.1 出題範囲の全体像
  - 12.2 頻出テーマの最終確認

    - 12.3 本番の解き方
    - 12.4 学習チェックリスト
    - 12.5 総仕上げ問題
- 付録A 練習問題の解答
    - 第1章
    - 第2章
    - 第3章
    - 第4章
    - 第5章
    - 第6章
    - 第7章
    - 第8章
    - 第9章
    - 第10章
    - 第11章
    - 第12章
- 付録B コマンド早見表
    - ファイル・ディレクトリ
    - テキスト処理
    - プロセス・システム
    - ネットワーク
    - パッケージ・ストレージ
    - セキュリティ
- 付録C 用語集
- 付録D 参考情報
    - 公式情報
    - 学習に役立つ情報
    - 免責

第1章 Linuxのインストールと仮想マシン・コンテナの利用

この章で学ぶこと

- Linux のインストール手順と、UEFI/BIOS のブート設定
- システムの起動・停止と、SSH による接続・切断
- ハイパーバイザーとコンテナの違い、仮想マシン／コンテナの操作
- systemd のブートターゲットとブートプロセス
- プロセスの生成・監視・終了、ジョブ管理
- X Window System の基本構成

1.1 Linux のインストール

Linux のインストールは、大きく次の流れで進みます。

1. インストールメディア（DVD/USB）または ISO イメージを用意する。
2. UEFI/BIOS の設定で、ブートメディアの優先順位を変更する。
3. インストーラを起動し、言語・タイムゾーン・キーボードを設定する。
4. ディスクのパーティションを構成する。
5. パッケージグループ（最小構成／GUI 付きなど）を選択する。
6. root パスワードと一般ユーザーを作成する。
7. インストールを実行し、再起動する。

UEFI と BIOS

最近の PC は **UEFI** が主流です。UEFI は従来の BIOS より大きなディスクを扱え、**GPT** パーティションテーブルと組み合わせて使われます。UEFI でブートするには、ディスク上に **ESP（EFI System Partition）** が必要です。ESP は FAT32 でフォーマットされ、通常 `/boot/efi` にマウントされます。

項目	BIOS（レガシー）	UEFI
パーティションテーブル	MBR が中心	GPT が中心
ブートローダ	MBR に格納	ESP 内の EFI アプリケーション
ディスク容量の制限	MBR は2TBまで	大容量ディスクに対応
セキュアブート	なし	対応（Secure Boot）

インストール時には、ブートメディアの優先順位を一時的に変更します。UEFI の画面では「Boot Order」「Boot Priority」などの項目を探します。

パッケージグループとタイムゾーン

インストーラでは、用途に応じてパッケージグループを選びます。

- **最小構成（Minimal）**: サーバー用途。必要なパッケージだけを入れる。
- **GUI 付き**: デスクトップ環境を含む。
- **開発ツール**: コンパイラやライブラリを含む。

タイムゾーンは `Asia/Tokyo` を選びます。GUI か CUI かは、用途と運用方針で決めます。サーバーでは CUI が一般的です。

1.2 起動・停止と SSH 接続

起動と停止

Linux の起動・停止には、次のコマンドを使います。

```
shutdown -h now      # 今すぐ停止（halt）
shutdown -r now      # 今すぐ再起動（reboot）
shutdown -h +10      # 10分後に停止
shutdown -c          # 予定されたシャットダウンを取消
reboot               # 再起動
halt                 # 停止
poweroff             # 電源オフ
```

`shutdown` はシステムを安全に停止します。`halt` と `poweroff` の違いは、電源管理の動作にあります。試験では「システムを停止するコマンド」として `shutdown -h now`、`halt`、`poweroff` が選べることを覚えておきましょう。

SSH による接続

遠隔から Linux に接続するには SSH を使います。

```
ssh user@192.168.1.10
ssh -p 2222 user@server.example.com
exit        # または logout、Ctrl-D
```

初回接続時には、サーバーのホスト鍵が `~/.ssh/known_hosts` に保存されます。指紋の確認を求められたら、管理者から提示された値と照合します。

公開鍵認証を使う場合、クライアントで鍵ペアを作成します。

```
ssh-keygen -t ed25519 -C "user@example.com"
# ~/.ssh/id_ed25519       （秘密鍵。600）
# ~/.ssh/id_ed25519.pub   （公開鍵。644）
ssh-copy-id user@server  # 公開鍵をサーバーの authorized_keys に登録
```

サーバー側では、`~/.ssh/authorized_keys` に公開鍵を登録します。パーミッションが緩いと SSH が鍵を拒否するため、次の権限を守ります。

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
chmod 600 ~/.ssh/id_ed25519
```

試験では `id_rsa`／`id_rsa.pub` という従来の RSA 鍵の名前も出題されます。現在は Ed25519 が推奨ですが、名称は覚えておきましょう。

1.3 仮想マシンとコンテナ

ハイパーバイザー

仮想マシン（VM）は、**ハイパーバイザー**と呼ばれるソフトウェアがハードウェアを仮想化し、その上でゲスト OS を動かします。

種類	説明	例
タイプ1（ベアメタル）	ハードウェア上で直接動作	KVM、VMware ESXi、Hyper-V
タイプ2（ホスト型）	ホスト OS 上で動作	VirtualBox、VMware Workstation

Linux の KVM はカーネルモジュールとして仮想化機能を提供し、`libvirt` と `virsh` で管理できます。CPU の仮想化支援機能（Intel VT-x／AMD-V）が有効になっている必要があります。

```
virsh list --all          # 仮想マシン一覧
virsh start web01         # 起動
virsh shutdown web01      # 正常停止
virsh destroy web01       # 強制停止
virsh console web01       # コンソール接続
```

コンテナ

コンテナは、ホスト OS のカーネルを共有し、**名前空間（namespaces）**と**cgroups**によってプロセスを隔離します。仮想マシンのようにゲスト OS を持たないため、起動が速く、リソース消費が少ないのが特徴です。

観点	仮想マシン	コンテナ
カーネル	ゲストごとに持つ	ホストと共有
起動時間	比較的遅い	非常に速い
隔離レベル	強い	ホストカーネルを共有するため比較的弱い
リソース消費	大きい	小さい
用途	異なる OS、強い隔離	アプリケーションの軽量な実行環境

Docker の基本操作は次のとおりです。

```
docker run -d --name web -p 8080:80 nginx  # イメージからコンテナを起動
docker ps                                  # 実行中コンテナー一覧
docker ps -a                               # 停止中も含めて表示
docker exec -it web bash                   # コンテナ内でシェルを実行
docker stop web                            # 停止
docker start web                           # 起動
docker rm web                              # 削除
docker images                              # イメージー覧
```

`-p 8080:80` は「ホストの8080番ポートをコンテナの80番ポートに転送する」という意味です。

1.4 ブートプロセスと systemd

Linux の起動は、おおまかに次の順序で進みます。

1. UEFI/BIOS が起動し、ブートデバイスを探す。
2. ブートローダ（GRUB2 など）がカーネルと initramfs を読み込む。
3. カーネルが初期化され、initramfs から必要なドライバを読み込む。
4. カーネルが PID 1 として **systemd** を起動する。
5. systemd がデフォルトのターゲットまでサービスを起動する。

ブートターゲット

systemd では、従来のランレベルに代えて**ターゲット**を使います。

ターゲット	意味
`poweroff.target`	停止
`rescue.target`	シングルユーザーモード相当（最小限の修復環境）
`multi-user.target`	複数ユーザー・CUI（サーバーの標準）
`graphical.target`	GUI を含むマルチユーザー
`reboot.target`	再起動

```
systemctl get-default                    # 現在のデフォルトターゲット
systemctl set-default multi-user.target  # デフォルトを CUI に変更
systemctl isolate rescue.target          # 一時的に rescue へ切り替え
systemctl reboot                         # 再起動
systemctl poweroff                       # 停止
```

`isolate` は現在のターゲットを即座に切り替えます。試験では `systemctl set-default` と `systemctl isolate` の違いが問われます。

1.5 プロセスの生成・監視・終了

プロセスの表示

```
ps aux          # すべてのプロセスを詳細表示
ps -ef          # 親子関係を含む形式
ps -ef --forest # プロセスの木構造
top             # リアルタイム監視
pstree -p       # プロセスツリーと PID
uptime          # 稼働時間・負荷平均
pgrep -u user   # ユーザー user のプロセスを PID で検索
```

ps aux の主な列は、USER（実行ユーザー）、PID（プロセス ID）、%CPU、%MEM、VSZ（仮想メモリ）、RSS（物理メモリ）、STAT（状態）、COMMAND です。STAT の R は実行中、S は待機中、Z はゾンビ、T は停止を表します。

ジョブ管理

```
sleep 300 &     # バックグラウンドで実行
jobs            # ジョブ一覧
fg %1           # ジョブ1をフォアグラウンドへ
bg %1           # ジョブ1をバックグラウンドで再開
```

ログアウト後も処理を続けたい場合は、nohup や screen、tmux を使います。

```
nohup long_task.sh > out.log 2>&1 &
screen -S work
tmux new -s work
```

シグナル

```
kill 1234          # PID 1234 に TERM（15）を送る
kill -9 1234       # KILL（強制終了）
kill -HUP 1234     # HUP（設定再読み込みなど）
pkill -f httpd     # コマンド名や引数で一致するプロセスに送信
killall httpd      # 名前が一致する全プロセスに送信
```

SIGTERM（15）は正常終了を促すシグナル、SIGKILL（9）は強制終了で捕捉できません。まず TERM を送り、応答がない場合に KILL を使うのが基本です。

1.6 デスクトップ環境（X Window System）

X Window System（X11）は、GUI を実現するための基本的な仕組みです。

構成要素	役割
X サーバー	画面・キーボード・マウスなどの入出力を管理
X クライアント	アプリケーション。X サーバーに描画を依頼
ディスプレイマネージャ	ログイン画面を提供（GDM、LightDM など）
ウィンドウマネージャ	ウィンドウの枠・配置・移動を管理
統合デスクトップ環境	GNOME、KDE Plasma、Xfce など

`DISPLAY` 環境変数は、接続先の X サーバーを指定します（例: `:0`、`localhost:10.0`）。`xauth` は X サーバーへのアクセス認証を管理します。SSH の X11 転送を使うと、リモートの GUI アプリを手元に表示できます。

```
ssh -X user@server    # X11 転送を有効にして接続
ssh -Y user@server    # 信頼された X11 転送
echo $DISPLAY
startx                # X を手動で起動（環境による）
```

本章のまとめ

- インストールでは UEFI/BIOS、パーティション、パッケージグループ、タイムゾーンを確認する。
- 停止は `shutdown -h now`、再起動は `shutdown -r now`／`reboot`。
- SSH の鍵は `ssh-keygen` で作り、`authorized_keys` と `known_hosts` の役割を区別する。
- 仮想マシンはゲスト OS を持ち、コンテナはホストのカーネルを共有する。
- systemd のターゲットは `multi-user.target` と `graphical.target` を中心に覚える。
- プロセスは `ps`／`top`、ジョブは `jobs`／`fg`／`bg`、終了は `kill` を使う。

練習問題

練習問題 1.1　UEFI 環境で Linux を起動するために必要なパーティションはどれか。

(a) `/home`　(b) ESP（EFI System Partition）　(c) スワップ　(d) `/var`

練習問題 1.2　10分後にシステムを停止するコマンドとして正しいものはどれか。

(a) `shutdown -r +10`　(b) `shutdown -h +10`　(c) `halt +10`　(d) `reboot 10`

練習問題 1.3　コンテナの説明として最も適切なものはどれか。

(a) ゲスト OS ごとに独立したカーネルを持つ　(b) ホスト OS のカーネルを共有し、名前空間で隔離する　(c) 必ず GUI が必要である　(d) 仮想マシンより起動に時間がかかる

練習問題 1.4　デフォルトのブートターゲットを CUI のマルチユーザーモードに変更するコマンドはどれか。

(a) `systemctl isolate multi-user.target`　(b) `systemctl set-default multi-user.target`　(c) `systemctl enable multi-user.target`　(d) `systemctl start rescue.target`

練習問題 1.5　実行中のプロセスに強制終了のシグナルを送るコマンドはどれか。

(a) `kill -9 PID`　(b) `kill -15 PID`　(c) `kill -1 PID`　(d) `kill -0 PID`

第2章 ファイル・ディレクトリの操作と管理

この章で学ぶこと

- Linux のパーミッションと所有者の考え方
- `chmod`／`chown`／`chgrp`／`umask` の使い方
- ファイル・ディレクトリの基本操作とアーカイブ
- ハードリンクとシンボリックリンクの違い
- FHS（ファイルシステム階層標準）とファイル検索

2.1 パーミッションと所有者

`ls -l` の出力は、次のような構造です。

```
-rw-r--r-- 1 user group  1234 Apr  1 10:00 report.txt
drwxr-xr-x 2 user group  4096 Apr  1 10:00 docs/
```

先頭の10文字のうち、1文字目はファイル種別（`-` 通常ファイル、`d` ディレクトリ、`l` シンボリックリンク、`c` キャラクタデバイス、`b` ブロックデバイス）、残り9文字は3文字ずつ「所有者」「グループ」「その他」の権限を表します。

記号	意味	数値
`r`	読み取り	4
`w`	書き込み	2
`x`	実行（ディレクトリでは移動・中身の参照）	1
`-`	権限なし	0

たとえば `rwxr-xr--` は、所有者が 7、グループが 5、その他が 4 で、**754** と表せます。

```
chmod 754 report.txt
chmod u+x script.sh          # 所有者に実行権を追加
chmod g-w,o-r report.txt     # グループの書き込みとその他の読み取りを削除
chmod -R 755 public_html     # 再帰的に変更
chown user:group report.txt  # 所有者とグループを変更
chgrp developers report.txt  # グループだけ変更
```

特殊なパーミッション

ビット	数値	意味
SUID	4000	実行時にファイル所有者の権限で動く（例: `/usr/bin/passwd`）

ビット	数値	意味
SGID	2000	実行時にグループの権限で動く。ディレクトリでは新規ファイルが親ディレクトリのグループを継承
スティッキービット	1000	ディレクトリ内のファイルを所有者以外が削除できない（例: /tmp）

```
chmod 4755 /usr/local/bin/tool   # SUID
chmod 2775 /shared               # SGID
chmod 1777 /tmp                  # スティッキービット
ls -ld /tmp                      # drwxrwxrwt ...
```

SUID が設定されたファイルは、セキュリティ監査の対象になります。`find / -perm -4000 -type f 2>/dev/null` で検索できます。

umask

`umask` は、新規ファイル・ディレクトリ作成時に**剥奪する権限**を指定します。ファイルの基準は 666、ディレクトリの基準は 777 です。

```
umask          # 例: 0022
umask 027      # グループの書き込み、その他の全権限を剥奪
```

`umask 022` の場合、ファイルは `666 - 022 = 644`、ディレクトリは `777 - 022 = 755` になります。`umask 027` ならファイルは 640、ディレクトリは 750 です。

2.2 基本的なファイル管理

```
ls -la               # 隠しファイルを含めて詳細表示
touch newfile        # 空ファイルを作成／タイムスタンプ更新
cp file1 file2       # コピー
cp -r dir1 dir2      # ディレクトリを再帰コピー
mv file1 dir/        # 移動
mv oldname newname   # 名前変更
rm file              # 削除
rm -r dir            # ディレクトリを再帰削除
rmdir emptydir       # 空ディレクトリを削除
mkdir -p a/b/c       # 親ディレクトリもまとめて作成
file report.txt      # ファイル種別を判定
```

ワイルドカード

記号	意味	例
*	0文字以上の任意文字	*.txt

記号	意味	例
`?`	任意の1文字	`file?.log`
`[abc]`	a、b、c のいずれか	`file[123].txt`
`[a-z]`	範囲	`[a-z]*.conf`
`[!abc]`	指定以外	`[!0-9]*`

アーカイブと圧縮

```
tar -cvf archive.tar dir/          # アーカイブ作成
tar -xvf archive.tar               # 展開
tar -tvf archive.tar               # 内容一覧
tar -czvf archive.tar.gz dir/      # gzip 圧縮
tar -xzvf archive.tar.gz           # gzip 展開
tar -cjvf archive.tar.bz2 dir/     # bzip2
tar -cJvf archive.tar.xz dir/      # xz
gzip file                          # file.gz を作成（元ファイルは削除）
gunzip file.gz                     # 展開
xz -9 file                         # 高圧縮
dd if=/dev/zero of=file bs=1M count=100  # 100MB のゼロファイル
```

`tar` のオプションは、`c`（作成）、`x`（展開）、`t`（一覧）、`v`（詳細）、`f`（ファイル指定）、`z`（gzip）、`j`（bzip2）、`J`（xz）を組み合わせます。

2.3 ハードリンクとシンボリックリンク

Linux のファイルは、**inode** と呼ばれる管理情報で識別されます。ファイル名は inode への参照です。

- **ハードリンク**: 同じ inode を指す別の名前。元ファイルを削除しても、リンクが残っていればデータは残る。異なるファイルシステム間では作成できない。ディレクトリには作成できない（通常）。
- **シンボリックリンク**: リンク先のパス名を保持する特別なファイル。異なるファイルシステム間でも作成できる。リンク先が削除されると壊れたリンクになる。

```
ln original hardlink       # ハードリンク
ln -s /var/www/html link   # シンボリックリンク
ls -li                     # inode 番号も表示
ls -l                      # シンボリックリンクは l で始まり、-> でリンク先を表示
readlink link              # リンク先を表示
```

ハードリンクは inode 番号が元ファイルと同じになります。シンボリックリンクは別の inode を持ち、リンク先のパスを内容として保持します。

2.4 FHS とファイル検索

FHS（Filesystem Hierarchy Standard）は、Linux のディレクトリ構造の標準です。試験では「どのファイルがどこにあるか」がよく問われます。

ディレクトリ	主な内容
/	ルート。すべての起点
/bin	基本コマンド（最近は /usr/bin へのリンクが多い）
/sbin	システム管理用コマンド
/etc	システム設定ファイル
/home	ユーザーのホームディレクトリ
/root	root のホームディレクトリ
/var	可変データ（ログ、スプール、キャッシュ）
/tmp	一時ファイル。再起動で消えることが多い
/usr	ユーザー向けのプログラム・ライブラリ・共有データ
/usr/local	管理者が独自にインストールしたソフトウェア
/opt	追加パッケージ
/boot	カーネル、initramfs、GRUB の設定
/dev	デバイスファイル
/proc	カーネル・プロセスの情報（仮想ファイルシステム）
/sys	カーネル・デバイスの情報（sysfs）
/run	実行時の一時データ
/media	リムーバブルメディアの自動マウント先
/mnt	手動マウントの一時的な場所
/srv	サービス提供用データ

ファイル検索

```
find /var/log -name "*.log"          # 名前で検索
find /home -type f -size +10M        # 10MB 超の通常ファイル
find / -user alice -mtime -7         # alice 所有で7日以内に更新
find /tmp -type f -delete            # 検索して削除
locate httpd.conf                    # データベースから高速検索
sudo updatedb                        # locate のデータベース更新
which ls                             # PATH 上の実行ファイルを表示
whereis ls                           # バイナリ・マニュアル等の場所
type cd                              # コマンドの種類（組み込みか等）
```

`find` は実際のファイルシステムを走査するため時間がかかりますが、常に最新の結果を返します。`locate` は事前に作ったデータベースを使うため高速ですが、`updatedb` を実行するまで新規ファイルは反映されません。

本章のまとめ

- パーミッションは `rwx` と数値（4・2・1）の両方で理解する。
- SUID・SGID・スティッキービットは用途と代表例を覚える。
- `umask` は「剥奪する権限」であり、ファイル666／ディレクトリ777から引く。
- ハードリンクは inode を共有し、シンボリックリンクはパスを保持する。
- `find` は条件指定が豊富。`locate` は高速だがデータベース依存。
- FHS の主要ディレクトリの役割を表で整理する。

練習問題

練習問題 2.1　`umask` が 027 のとき、新規に作成される通常ファイルの権限はどれか。

(a) 640　(b) 644　(c) 750　(d) 755

練習問題 2.2　スティッキービットが設定されたディレクトリとして代表的なものはどれか。

(a) `/etc`　(b) `/tmp`　(c) `/boot`　(d) `/root`

練習問題 2.3　ハードリンクについて正しい説明はどれか。

(a) 異なるファイルシステム間でも作成できる　(b) 元ファイルと同じ inode を共有する　(c) リンク先のパス名を内容として持つ　(d) ディレクトリにのみ作成できる

練習問題 2.4　`/var/log` 以下から7日以内に更新された通常ファイルを検索するコマンドはどれか。

(a) `find /var/log -type f -mtime -7`　(b) `find /var/log -type d -mtime 7`　(c) `locate /var/log -mtime -7`　(d) `grep -r -mtime -7 /var/log`

練習問題 2.5　システム設定ファイルが置かれる標準的なディレクトリはどれか。

(a) `/var`　(b) `/usr`　(c) `/etc`　(d) `/opt`

第3章 GNUとUnixのコマンド

この章で学ぶこと

- シェル変数・環境変数とコマンドラインの基本操作
- テキストストリームを処理するフィルタコマンド
- 標準入力・標準出力・標準エラー出力とリダイレクト
- パイプ、xargs、tee の使い方
- 正規表現と grep／sed
- vi の基本操作

3.1 コマンドラインの操作

変数

シェル変数は現在のシェルだけで有効、環境変数は子プロセスにも引き継がれます。

```
name=Alice          # シェル変数（= の前後に空白を入れない）
echo "$name"
export name         # 環境変数にする
env                 # 環境変数の一覧
set                 # シェル変数を含む一覧
unset name          # 変数を削除
echo $PATH
export PATH="$PATH:/usr/local/bin"
```

引用符

記号	動作
'...'	中身をすべて文字どおり扱う
"..."	$変数、$(コマンド)、`コマンド` を展開する
\	次の1文字をエスケープする

```
echo '$HOME'   # $HOME と表示
echo "$HOME"   # /home/user などと表示
```

コマンド置換と履歴

```
today=$(date +%F)
echo "Today is $today"
history            # コマンド履歴
!123               # 履歴番号123を実行
!!                 # 直前のコマンド
man ls             # マニュアル表示
man 5 passwd       # セクション5（設定ファイル）を表示
```

~/.bash_history に履歴が保存されます。HISTSIZE や HISTFILESIZE で件数を制御できます。

3.2 フィルタによるテキスト処理

コマンド	主な用途
cat	ファイル内容を表示・連結
head / tail	先頭／末尾の行を表示（-n、tail -f）
cut	列を切り出す（-d 区切り、-f フィールド）
sort	並べ替え（-n 数値、-r 逆順、-k キー）
uniq	連続する重複行を除去（-c 件数、-d 重複のみ）
wc	行数・単語数・バイト数（-l、-w、-c）
tr	文字の置換・削除
sed	ストリームエディタ。置換・削除・抽出
nl	行番号を付ける
od	バイナリ・8進・16進で表示
paste	ファイルを横に結合
join	共通キーでファイルを結合
split	ファイルを分割
expand / unexpand	タブと空白の変換
fmt	テキストの折り返し
pr	印刷用に整形
less	ページャ（/ 検索、q 終了）

```
head -n 20 access.log
tail -n 50 access.log
tail -f /var/log/syslog
cut -d: -f1 /etc/passwd              # ユーザー名だけ
sort -t: -k3 -n /etc/passwd          # UID で数値ソート
uniq -c sorted.txt                   # 重複件数
wc -l file.txt
tr 'a-z' 'A-Z' < file.txt
```

3.3 ストリーム・パイプ・リダイレクト

Linux のプロセスには、標準入力（fd 0）、標準出力（fd 1）、標準エラー出力（fd 2）があります。

記法	意味
`>`	標準出力をファイルに上書き
`>>`	標準出力をファイルに追記
`2>`	標準エラー出力をファイルにリダイレクト
`2>&1`	標準エラー出力を標準出力に合流
`&>`	標準出力と標準エラー出力の両方をファイルへ（Bash）
`<`	標準入力をファイルから受け取る
`<<`	ヒアドキュメント
`|`	パイプ。前の出力を次の入力へ
`tee`	出力を画面とファイルの両方へ
`xargs`	標準入力をコマンドの引数に変換

```
command > out.txt 2> err.txt
command > out.txt 2>&1
command &> all.txt
cat <<EOF > config.txt
server=example.com
port=80
EOF
ps aux | grep sshd
find . -name "*.tmp" -print0 | xargs -0 rm -f
command | tee log.txt
```

`2>&1` は「ファイルディスクリプタ2を、現在のファイルディスクリプタ1の指す先へ複製する」という意味です。順序が重要で、`command 2>&1 > out.txt` と `command > out.txt 2>&1` では結果が異なります。

3.4 正規表現と grep／sed

基本正規表現（BRE）

記号	意味
`.`	任意の1文字
`*`	直前の文字の0回以上の繰り返し
`^`	行頭
`$`	行末
`[abc]`	a、b、c のいずれか
`[^abc]`	a、b、c 以外
`[a-z]`	範囲
`\`	メタ文字のエスケープ

拡張正規表現（ERE）では、`+`（1回以上）、`?`（0回または1回）、`|`（OR）、`()`（グループ）、`{n,m}`（回数）が使えます。

```
grep 'error' /var/log/syslog
grep -i 'error' log.txt          # 大文字小文字を無視
grep -v '^#' config.txt          # コメント行以外
grep -n 'root' /etc/passwd       # 行番号付き
grep -r 'Listen' /etc/httpd/     # 再帰検索
grep -E 'error|warning' log.txt  # 拡張正規表現
grep -c 'Failed' auth.log        # 一致行数
egrep '^(error|warning)' log.txt
fgrep 'a.b' file.txt             # 正規表現を使わない固定文字列
```

`sed` はストリームエディタです。

```
sed 's/old/new/' file.txt        # 各行の最初の old を new に
sed 's/old/new/g' file.txt       # すべて置換
sed -i 's/old/new/g' file.txt    # ファイルを直接編集
sed -n '1,10p' file.txt          # 1〜10行目を表示
sed '/^#/d' config.txt           # コメント行を削除
sed -e 's/a/b/' -e 's/c/d/' file.txt
```

3.5 vi エディタ

`vi` には**コマンドモード**と**挿入モード**があります。起動時はコマンドモードです。

操作	キー
挿入モードへ（カーソル位置）	`i`
挿入モードへ（カーソルの次）	`a`
挿入モードへ（下に新しい行）	`o`
コマンドモードへ戻る	`Esc`
保存して終了	`:wq` または `ZZ`
保存せず終了	`:q!`
強制保存	`:w!`
カーソル移動	`h j k l`
1行削除	`dd`
1行コピー	`yy`
貼り付け	`p`
検索	`/word`、`?word`
次の検索結果	`n`
置換	`:%s/old/new/g`

標準エディタを変更するには `EDITOR` 環境変数を使います。

```
export EDITOR=vim
export EDITOR=nano
```

本章のまとめ

- シェル変数と環境変数の違い、`export` の役割を理解する。
- フィルタは「入力→加工→出力」の部品。組み合わせて使う。
- `>` は上書き、`>>` は追記、`2>&1` はエラーを標準出力へ合流。
- `xargs` は標準入力を引数に変換。`tee` は画面とファイルの両方へ。
- 正規表現は BRE と ERE の違いを意識する。
- `vi` はモード切替、保存・終了、検索・置換を確実に覚える。

練習問題

練習問題 3.1　標準エラー出力を標準出力に合流させる記述はどれか。

(a) `2>&1`　(b) `1>&2`　(c) `2>1`　(d) `&>2`

練習問題 3.2　`/etc/passwd` の1列目（ユーザー名）だけを表示するコマンドはどれか。

(a) `cut -d: -f1 /etc/passwd`　(b) `cut -f1 /etc/passwd`　(c) `awk -F, '{print $1}' /etc/passwd`　(d) `head -1 /etc/passwd`

練習問題 3.3　ファイル内のすべての `old` を `new` に置換し、直接ファイルを更新するコマンドはどれか。

(a) `sed 's/old/new/' file`　(b) `sed -i 's/old/new/g' file`　(c) `grep -v old file`　(d) `tr old new < file`

練習問題 3.4　`find` の結果を `rm` の引数として安全に渡すコマンドはどれか。

(a) `find . -name "*.tmp" | rm`　(b) `find . -name "*.tmp" -print0 | xargs -0 rm`　(c) `find . -name "*.tmp" > rm`　(d) `xargs find . -name "*.tmp" | rm`

練習問題 3.5　`vi` で変更を保存せずに終了する操作はどれか。

(a) `:wq`　(b) `ZZ`　(c) `:q!`　(d) `:w!`

第4章 リポジトリとパッケージ管理

この章で学ぶこと

- パッケージ管理の基本概念（リポジトリ、依存関係）
- Debian 系の `apt`／`dpkg` と Red Hat 系の `yum`／`rpm`
- パッケージの検索・情報表示・インストール・削除
- パッケージが提供するファイル、ファイルを含むパッケージの調べ方

4.1 パッケージ管理の基礎

Linux のソフトウェアは、**パッケージ**単位で配布されます。パッケージにはプログラム本体、設定ファイル、マニュアル、依存関係の情報が含まれます。パッケージをまとめて公開している場所を**リポジトリ**と呼びます。

パッケージマネージャの主な役割は次のとおりです。

- リポジトリからパッケージを取得する。
- 依存関係を解決する。
- インストール・アップグレード・削除を行う。
- パッケージデータベースを管理する。

系統	高レベルツール	低レベルツール	パッケージ形式
Debian/Ubuntu	`apt`、`apt-get`	`dpkg`	`.deb`
Red Hat/Fedora	`yum`、`dnf`	`rpm`	`.rpm`

高レベルツールはリポジトリと依存関係を扱い、低レベルツールは個々のパッケージファイルを直接扱います。

4.2 apt によるパッケージ管理

```
sudo apt update                 # リポジトリ情報を更新
sudo apt upgrade                # インストール済みパッケージを更新
sudo apt install nginx          # インストール
sudo apt remove nginx           # 削除（設定ファイルは残す）
sudo apt purge nginx            # 設定ファイルも削除
sudo apt search editor          # 検索
apt show nginx                  # パッケージ情報
apt list --installed            # インストール済み一覧
sudo apt autoremove             # 不要な依存パッケージを削除
```

`apt-cache` はパッケージ情報の検索に使います。

```
apt-cache search web server
apt-cache show nginx
apt-cache depends nginx          # 依存パッケージ
apt-cache rdepends nginx         # 逆依存
apt-cache policy nginx           # 候補バージョンと優先度
```

`apt-file` は「あるファイルを含むパッケージ」を検索します。

```
sudo apt-file update
apt-file search bin/nginx
apt-file list nginx
```

リポジトリの設定は `/etc/apt/sources.list` と `/etc/apt/sources.list.d/` にあります。Ubuntu では `deb` 行に URL、ディストリビューション、コンポーネント（main、universe など）を指定します。

4.3 dpkg による Debian パッケージ管理

`dpkg` は `.deb` ファイルを直接扱います。依存関係は自動解決しないため、必要に応じて `apt` を使います。

```
sudo dpkg -i package.deb         # インストール
sudo dpkg -r package             # 削除（設定は残す）
sudo dpkg -P package             # 完全削除
dpkg -l                          # インストール済み一覧
dpkg -l nginx                    # 特定パッケージの状態
dpkg -L nginx                    # パッケージがインストールしたファイル一覧
dpkg -S /usr/sbin/nginx          # ファイルを含むパッケージを検索
dpkg -s nginx                    # パッケージの状態・情報
sudo dpkg --configure -a         # 未設定パッケージを設定
sudo dpkg-reconfigure tzdata     # パッケージの再設定
```

`dpkg -l` の先頭2文字は状態を示します。`ii` は「インストール済みで設定済み」、`rc` は「削除済みだが設定ファイルが残っている」状態です。

4.4 yum によるパッケージ管理

```
sudo yum install httpd          # インストール
sudo yum remove httpd           # 削除
sudo yum update                 # すべて更新
sudo yum update httpd           # 特定パッケージを更新
yum search web server           # 検索
yum info httpd                  # 情報表示
yum list installed              # インストール済み一覧
yum provides "*/bin/ls"         # ファイルを提供するパッケージ
yum deplist httpd               # 依存関係
sudo yum clean all              # キャッシュ削除
sudo yum repolist               # 有効なリポジトリー覧
```

リポジトリの設定は /etc/yum.repos.d/ 以下の .repo ファイルと /etc/yum.conf にあります。.repo ファイルには [repo-id]、name、baseurl、enabled、gpgcheck などを記述します。

```
[epel]
name=Extra Packages for Enterprise Linux
baseurl=https://download.example.org/epel/$releasever/$basearch/
enabled=1
gpgcheck=1
```

yumdownloader はパッケージをダウンロードするだけで、インストールはしません。

```
yumdownloader httpd
yumdownloader --resolve httpd     # 依存パッケージも取得
```

4.5 rpm による RPM パッケージ管理

```
sudo rpm -ivh package.rpm       # インストール（i: install, v: verbose, h: hash）
sudo rpm -Uvh package.rpm       # アップグレード（未インストールならインストール）
sudo rpm -e package             # 削除
rpm -qa                         # インストール済み全パッケージ
rpm -qi httpd                   # 詳細情報
rpm -ql httpd                   # 含まれるファイル一覧
rpm -qf /usr/sbin/httpd         # ファイルを含むパッケージ
rpm -qp package.rpm             # 未インストールの rpm ファイルを照会
rpm -V httpd                    # ファイルの改ざん・欠落を検証
rpm --import RPM-GPG-KEY        # GPG 鍵をインポート
```

rpm -V は、パッケージのファイルがインストール時から変更されていないかを検証します。出力の記号（S サイズ、M モード、5 ダイジェスト、T タイムスタンプなど）で差異を確認します。

4.6 どちらを使うべきか

目的	Debian 系	Red Hat 系
リポジトリからインストール	`apt install`	`yum install`
ローカルのパッケージファイルをインストール	`dpkg -i`	`rpm -ivh`
ファイルを含むパッケージを検索	`apt-file search`／`dpkg -S`	`yum provides`／`rpm -qf`
パッケージのファイル一覧	`dpkg -L`	`rpm -ql`
インストール済み一覧	`dpkg -l`／`apt list --installed`	`rpm -qa`／`yum list installed`

試験では、コマンド名と「どの系統のツールか」をセットで覚えることが重要です。`apt` と `yum` は高レベル、`dpkg` と `rpm` は低レベルと整理しましょう。

本章のまとめ

- リポジトリはパッケージの公開場所。依存関係の解決は高レベルツールが担う。
- Debian 系は `apt`／`dpkg`、Red Hat 系は `yum`／`rpm`。
- `apt update` はリポジトリ情報の更新であり、システム更新ではない。
- ファイルからパッケージを調べる方法を両系統で覚える。
- `yumdownloader` はダウンロードのみ。`rpm -V` は検証。

練習問題

練習問題 4.1　Debian 系で、リポジトリ情報を更新するコマンドはどれか。

(a) `apt upgrade`　(b) `apt update`　(c) `dpkg -l`　(d) `yum update`

練習問題 4.2　`/usr/bin/ls` を含むパッケージを Debian 系で調べるコマンドはどれか。

(a) `dpkg -L coreutils`　(b) `dpkg -S /usr/bin/ls`　(c) `apt-file list coreutils`　(d) `dpkg -s coreutils`

練習問題 4.3　Red Hat 系で、ファイルを提供するパッケージを検索するコマンドはどれか。

(a) `rpm -qf`　(b) `yum provides`　(c) `yum info`　(d) `rpm -V`

練習問題 4.4　インストールせずに RPM パッケージをダウンロードするコマンドはどれか。

(a) `yum install`　(b) `yumdownloader`　(c) `rpm -ivh`　(d) `yum deplist`

練習問題 4.5　`dpkg -l` の状態 `rc` の意味はどれか。

(a) インストール済みで設定済み　(b) 削除済みだが設定ファイルが残っている　(c) インストール中　(d) 壊れている

第5章 ハードウェア、ディスク、パーティション、ファイルシステム

この章で学ぶこと

- ハードウェア情報の取得とカーネルモジュールの管理
- HDD／SSD の違い、MBR と GPT、パーティションの考え方
- LVM の基本概念と操作
- ファイルシステムの作成・マウント・`/etc/fstab`
- スワップ領域の作成と有効化

5.1 ハードウェアの基礎

ハードウェア情報の取得

```
lspci                # PCI デバイス一覧
lsusb                # USB デバイス一覧
lsblk                # ブロックデバイスとパーティション
lscpu                # CPU 情報
free -h              # メモリ使用量
dmidecode -t memory  # メモリの詳細（要 root）
cat /proc/cpuinfo
cat /proc/meminfo
```

`/proc` はカーネルが提供する仮想ファイルシステムで、プロセスやハードウェアの情報をファイルとして見せます。`/sys` は sysfs で、デバイスやドライバの情報を提供します。`/dev` にはデバイスファイルが置かれます。

カーネルモジュール

```
lsmod                    # 読み込まれているモジュール
modinfo e1000            # モジュール情報
sudo modprobe e1000      # 依存関係を解決して読み込む
sudo insmod module.ko    # モジュールファイルを直接読み込む
sudo rmmod e1000         # アンロード
```

`modprobe` は依存関係を解決してくれますが、`insmod` は指定したファイルをそのまま読み込みます。設定は `/etc/modprobe.d/` に置きます。

`udev` はデバイスの検出時に `/dev` のデバイスファイルを作成・管理します。`dbus` はアプリケーション間のメッセージ通信を提供します。試験では「sysfs、udev、dbus の概念」が問われるため、役割を整理しておきましょう。

記憶装置の種類

種類	特徴
HDD	磁気ディスク。大容量・低コスト。シーク時間がある
SSD	フラッシュメモリ。高速・耐衝撃。書き込み回数に制限
光学ストレージ	CD/DVD/Blu-ray。読み取り専用の用途が多い

5.2 パーティションとディスクレイアウト

MBR と GPT

項目	MBR	GPT
最大ディスク	2TB	非常に大容量
パーティション数	基本4つ（拡張で拡張）	多数（通常128）
ブート	BIOS + MBR	UEFI + ESP
冗長性	なし	パーティションテーブルのバックアップあり

```
sudo fdisk -l                # パーティション一覧
sudo fdisk /dev/sdb          # MBR ディスクの編集
sudo gdisk /dev/sdb          # GPT ディスクの編集
sudo parted /dev/sdb print    # parted で表示
```

`fdisk` の主な操作は、`n`（新規）、`d`（削除）、`p`（表示）、`t`（種類変更）、`w`（書き込み）、`q`（終了）です。パーティション変更後は `partprobe` や再読み込みが必要な場合があります。

代表的なパーティション構成

マウントポイント	用途
`/`	ルートファイルシステム
`/boot`	カーネルとブートローダ
`/boot/efi`	UEFI の ESP
`/home`	ユーザーデータ
`/var`	ログ・スプールなど変化するデータ
`swap`	スワップ領域

サーバーでは、`/var` や `/home` を別パーティションに分けることで、ログ肥大化やユーザーデータ増加がルートを圧迫するのを防げます。

5.3 LVM

LVM（Logical Volume Manager）は、物理ディスクを柔軟に組み合わせて論理ボリュームを作る仕組みです。

用語	意味
PV（Physical Volume）	LVM で使う物理ボリューム（パーティションやディスク）
VG（Volume Group）	PV をまとめたプール
LV（Logical Volume）	VG から切り出した論理ボリューム

```
sudo pvcreate /dev/sdb                 # PV 作成
sudo vgcreate vgdata /dev/sdb          # VG 作成
sudo lvcreate -L 10G -n lvweb vgdata   # 10GB の LV 作成
sudo lvextend -L +5G /dev/vgdata/lvweb # 5GB 拡張
sudo resize2fs /dev/vgdata/lvweb       # ext4 のサイズ反映
sudo xfs_growfs /mnt/web               # XFS のサイズ反映
sudo lvcreate -s -L 2G -n snap /dev/vgdata/lvweb  # スナップショット
```

LVM の利点は、動的な拡張・縮小、スナップショット、複数ディスクの束ね上げです。欠点は、構成が複雑になり、障害時の復旧が難しくなることです。

5.4 ファイルシステムの作成とマウント

主なファイルシステム

ファイルシステム	特徴
ext4	Linux で広く使われる。ジャーナリング、拡張属性対応
XFS	大容量・高並列性能。拡張はできるが縮小はできない
Btrfs	スナップショット、サブボリューム、圧縮、チェックサム
VFAT	USB メモリなどで使われる。Windows と互換
tmpfs	メモリ上に作る一時ファイルシステム。再起動で消える

```
sudo mkfs.ext4 /dev/sdb1
sudo mkfs.xfs /dev/sdb1
sudo mkfs.vfat /dev/sdb1
sudo mkfs.btrfs /dev/sdb1
```

スワップ

```
sudo mkswap /dev/sdb2       # スワップ領域を初期化
sudo swapon /dev/sdb2       # 有効化
sudo swapon --show          # 確認
sudo swapoff /dev/sdb2      # 無効化
free -h                     # Swap 行を確認
```

/etc/fstab に swap として登録すると、起動時に自動で有効になります。

マウント

```
sudo mount /dev/sdb1 /mnt/data
sudo umount /mnt/data
mount | grep sdb1
findmnt
sudo blkid                    # UUID とラベルを確認
sudo mount UUID="xxxx-xxxx" /mnt/data
sudo mount LABEL="DATA" /mnt/data
```

umount は「アンマウント」であり、unmount ではありません。使用中のファイルシステムはアンマウントできないため、lsof や fuser で使用プロセスを確認します。

/etc/fstab

```
UUID=1234-ABCD  /          ext4  defaults        0 1
UUID=5678-EFGH  /boot      ext4  defaults        0 2
UUID=90AB-CDEF  /home      xfs   defaults        0 0
UUID=1111-2222  none       swap  sw              0 0
```

フィールド	意味
1	デバイス（UUID、LABEL、/dev/sdb1 など）
2	マウントポイント
3	ファイルシステム種別
4	マウントオプション（defaults、noatime、ro など）
5	dump の対象（通常0）
6	起動時の fsck 順序（ルートは1、その他は2、不要は0）

/etc/fstab を編集したら、sudo mount -a で構文とマウントを確認します。誤った設定は起動失敗につながるため注意が必要です。

本章のまとめ

- ハードウェア情報は lspci、lsusb、lsblk、lscpu などで取得する。

- モジュールは `lsmod`／`modprobe`／`insmod`／`rmmod` で管理する。
- MBR は2TB制限、GPT は UEFI と ESP が基本。
- LVM は PV → VG → LV の順に構成する。
- ext4 は拡張・縮小可能、XFS は拡張のみ。
- マウント設定は `/etc/fstab`。UUID を使うとデバイス名の変化に強い。

練習問題

練習問題 5.1　読み込まれているカーネルモジュールの一覧を表示するコマンドはどれか。

(a) `lsmod`　(b) `lspci`　(c) `lsblk`　(d) `modinfo`

練習問題 5.2　UEFI 環境で必要な ESP をマウントする標準的な場所はどれか。

(a) `/boot`　(b) `/boot/efi`　(c) `/efi`　(d) `/mnt/efi`

練習問題 5.3　LVM の構成要素を正しい順序で並べたものはどれか。

(a) LV → VG → PV　(b) PV → VG → LV　(c) VG → PV → LV　(d) PV → LV → VG

練習問題 5.4　XFS ファイルシステムについて正しい説明はどれか。

(a) 拡張も縮小も自由にできる　(b) 拡張はできるが縮小はできない　(c) スナップショットが標準機能である　(d) メモリ上に作られる

練習問題 5.5　`/etc/fstab` の6番目のフィールドの意味はどれか。

(a) dump の対象　(b) マウントオプション　(c) 起動時の fsck 順序　(d) ファイルシステム種別

第6章 シェルおよびスクリプト

この章で学ぶこと

- ログインシェルと非ログインシェルの設定ファイル
- 環境変数、PATH、alias、source の使い方
- Bash スクリプトの基本構文（変数、引数、条件分岐、ループ、関数）
- 戻り値と exit、デバッグ方法
- 安全で再利用可能なスクリプトの書き方

6.1 シェル環境のカスタマイズ

設定ファイルの読み込み

Bash は、起動の種類によって読み込む設定ファイルが異なります。

起動の種類	主な読み込みファイル
ログインシェル	/etc/profile → ~/.bash_profile → ~/.bash_login → ~/.profile の順で最初に見つかったもの
対話型の非ログインシェル	/etc/bash.bashrc → ~/.bashrc
非対話型シェル	通常は設定ファイルを読まない（BASH_ENV を除く）

ディストリビューションによって /etc/bashrc など名称が異なる場合があります。~/.bash_profile から ~/.bashrc を source する構成も一般的です。

```
source ~/.bashrc       # 現在のシェルに設定を反映
. ~/.bashrc            # source と同じ
alias ll='ls -alF'
alias grep='grep --color=auto'
export PATH="$HOME/bin:$PATH"
```

; は前のコマンドの結果に関係なく次を実行、&& は前が成功したときだけ次を実行、|| は前が失敗したときだけ次を実行します。

```
cd /tmp && rm -f workfile
test -f config || echo "設定ファイルがありません"
```

6.2 Bash スクリプトの基本

スクリプトの作成と実行

```
#!/bin/bash
# hello.sh
echo "Hello, world!"
```

```
chmod +x hello.sh
./hello.sh
bash hello.sh
```

先頭の `#!` は**シバン（shebang）**と呼ばれ、スクリプトを実行するインタプリタを指定します。`#!/bin/bash` は Bash で実行するという意味です。

変数と引数

```
name="Alice"          # = の前後に空白を入れない
echo "$name"
echo "${name}san"     # 変数名の境界を明示

echo "$0"             # スクリプト名
echo "$1"             # 第1引数
echo "$#"             # 引数の数
echo "$@"             # すべての引数（個別に展開）
echo "$*"             # すべての引数（1つの文字列として展開）
shift                 # 引数を1つずらす
echo "$?"             # 直前のコマンドの終了ステータス
```

`$@` は `"$@"` と引用すると各引数を個別に扱い、`$*` は1つの文字列として扱います。引数に空白が含まれる可能性がある場合は `"$@"` を使います。

終了ステータス

Linux のコマンドは、成功時に 0、失敗時に 0 以外を返します。

```
grep -q "error" log.txt
if [ $? -eq 0 ]; then
  echo "エラーが見つかりました"
fi
exit 1
```

`exit 0` は成功、`exit 1` は一般的な失敗を表します。

条件分岐

```
if [ -f /etc/passwd ]; then
  echo "ファイルがあります"
elif [ -d /etc ]; then
  echo "ディレクトリがあります"
else
  echo "どちらでもありません"
fi

case "$1" in
  start) echo "開始します" ;;
  stop)  echo "停止します" ;;
  *)     echo "使い方: $0 {start|stop}" ; exit 1 ;;
esac
```

テスト	意味
`-f file`	通常ファイルが存在
`-d dir`	ディレクトリが存在
`-e path`	存在する
`-r` / `-w` / `-x`	読み取り／書き込み／実行可能
`-z str`	文字列が空
`-n str`	文字列が空でない
`a -eq b`	数値が等しい
`a -ne b`	数値が等しくない
`a -lt b`	a が b より小さい
`str1 = str2`	文字列が等しい
`str1 != str2`	文字列が異なる

ループ

```
for file in *.txt; do
  echo "処理中: $file"
done

for i in $(seq 1 5); do
  echo "$i"
done

while read -r line; do
  echo "$line"
done < input.txt

count=0
while [ $count -lt 3 ]; do
  echo "$count"
  count=$((count + 1))
done
```

関数

```
greet() {
  local name="$1"
  echo "Hello, $name"
}

greet "Alice"
```

`local` を使うと関数内だけの変数になります。関数は `return` で終了ステータスを返せますが、文字列を返す場合は標準出力を使います。

安全なスクリプトの書き方

```
#!/bin/bash
set -euo pipefail
```

- `-e`: コマンドが失敗したら即座に終了
- `-u`: 未定義変数を参照したらエラー
- `-o pipefail`: パイプ内の失敗も検出

そのほか、`trap` で終了時の後処理を定義できます。

```
trap 'rm -f "$tmpfile"' EXIT
tmpfile=$(mktemp)
```

デバッグ

```
bash -x script.sh      # 実行するコマンドを表示
bash -v script.sh      # 読み込んだ行を表示
bash -n script.sh      # 構文チェックのみ
```

本章のまとめ

- ログインシェルと非ログインシェルで読み込む設定ファイルが異なる。
- `source`／`.` は現在のシェルに設定を反映する。
- `&&` は成功時、`||` は失敗時に次を実行。
- スクリプトはシバンで始め、実行権を付ける。
- 引数は `$1`、`$#`、`"$@"`、`shift` を使い分ける。
- `set -euo pipefail` と `trap` で安全なスクリプトにする。

練習問題

練習問題 6.1　現在のシェルに `~/.bashrc` の設定を反映するコマンドはどれか。

(a) `bash ~/.bashrc`　(b) `source ~/.bashrc`　(c) `exec ~/.bashrc`　(d) `cat ~/.bashrc`

練習問題 6.2　スクリプトの第1引数を表す変数はどれか。

(a) `$0`　(b) `$1`　(c) `$#`　(d) `$?`

練習問題 6.3　直前のコマンドの終了ステータスを表す変数はどれか。

(a) `$$`　(b) `$!`　(c) `$?`　(d) `$*`

練習問題 6.4　引数がすべて個別の文字列として展開されるようにする記述はどれか。

(a) `$*`　(b) `"$*"`　(c) `"$@"`　(d) `$@` を引用符なしで使う

練習問題 6.5　シェルスクリプトの構文チェックだけを行うコマンドはどれか。

(a) `bash -x script.sh`　(b) `bash -v script.sh`　(c) `bash -n script.sh`　(d) `bash -e script.sh`

第7章 ネットワークの基礎

この章で学ぶこと

- TCP/IP の基礎、IPv4／IPv6、サブネットと CIDR
- 主要なポート番号と TCP／UDP／ICMP の違い
- Linux のネットワーク設定（`ip`、`nmcli`、設定ファイル）
- 名前解決の仕組みと DNS クライアント設定
- ネットワーク障害の切り分けと調査コマンド

7.1 インターネットプロトコルの基礎

TCP/IP モデル

TCP/IP は、ネットワーク通信を4つの層に分けて考えます。

層	役割	主なプロトコル
アプリケーション層	アプリケーション間の通信	HTTP、SSH、DNS、SMTP
トランスポート層	端末間の通信、ポート番号	TCP、UDP
インターネット層	アドレスとルーティング	IP、ICMP
ネットワークインターフェース層	物理的な通信	Ethernet、ARP

IPv4 アドレスとサブネット

IPv4 アドレスは32ビットで、8ビットずつ4つに区切って表します（例: `192.168.1.10`）。ネットワーク部とホスト部を分けるのが**サブネットマスク**です。

CIDR	サブネットマスク	利用可能ホスト数（目安）
`/24`	255.255.255.0	254
`/25`	255.255.255.128	126
`/26`	255.255.255.192	62
`/30`	255.255.255.252	2
`/32`	255.255.255.255	1（単一ホスト）

プライベートアドレスは次の範囲です。

- `10.0.0.0/8`
- `172.16.0.0/12`（172.16.0.0〜172.31.255.255）
- `192.168.0.0/16`
- `127.0.0.0/8`（ループバック）

- `169.254.0.0/16`（APIPA／リンクローカル）

主要なポート番号

ポート	プロトコル	用途
22	TCP	SSH
25	TCP	SMTP
53	TCP/UDP	DNS
80	TCP	HTTP
123	UDP	NTP
443	TCP	HTTPS

TCP・UDP・ICMP

プロトコル	特徴	用途
TCP	コネクション型、再送・順序保証あり	HTTP、SSH、SMTP
UDP	コネクションレス型、高速だが信頼性なし	DNS、NTP、音声・動画
ICMP	エラー通知・診断	`ping`、`traceroute`

IPv6

IPv6 は128ビットのアドレス空間を持ちます。主な特徴は次のとおりです。

- アドレスを `:` で区切って16進数で表す（例: `2001:db8::1`）。
- `::` は連続する0を1回だけ省略できる。
- ブロードキャストがなく、マルチキャストとエニーキャストを使う。
- 近隣探索（NDP）と SLAAC による自動設定がある。
- リンクローカルアドレスは `fe80::/10`。
- グローバルユニキャストは `2000::/3`。
- ユニークローカルアドレスは `fc00::/7`。

7.2 ネットワーク設定

ip コマンド

```
ip addr show                    # アドレス一覧
ip addr add 192.168.1.10/24 dev eth0
ip link set eth0 up
ip link set eth0 down
ip route show                   # ルーティングテーブル
ip route add default via 192.168.1.1
ip route del default
ip neigh                        # ARP／近隣テーブル
```

従来の ifconfig、route、arp も試験範囲です。現在の Linux では ip 系のコマンドが推奨されます。

```
ifconfig eth0
ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up
route -n
route add default gw 192.168.1.1
```

NetworkManager

```
nmcli device status
nmcli connection show
nmcli connection up eth0
nmcli connection modify eth0 ipv4.addresses 192.168.1.10/24
nmcli connection modify eth0 ipv4.gateway 192.168.1.1
nmcli connection modify eth0 ipv4.method manual
nmcli connection up eth0
```

ifup／ifdown は設定ファイルに基づいてインターフェースを制御します。ディストリビューションによって設定ファイルの場所が異なります。

ディストリビューション	設定ファイルの例
Debian/Ubuntu（従来）	/etc/network/interfaces
RHEL 系	/etc/sysconfig/network-scripts/ifcfg-eth0
NetworkManager	nmcli で管理

ホスト名と hosts

```
hostname
hostnamectl set-hostname web01
cat /etc/hostname
cat /etc/hosts
```

/etc/hosts は、IP アドレスとホスト名の対応をローカルに定義します。DNS より先に参照されることが多いため、名前解決の順序は /etc/nsswitch.conf の hosts: 行で決まります。

7.3 ネットワークの問題解決

```
ping -c 4 8.8.8.8          # 疎通確認（ICMP）
ping6 -c 4 2001:4860:4860::8888
host example.com            # DNS 問い合わせ
dig example.com             # 詳細な DNS 問い合わせ
dig +short example.com
dig @8.8.8.8 example.com
ss -tuln                    # 待ち受けポート（TCP/UDP）
ss -tanp                    # 確立済み TCP とプロセス
netstat -tulnp              # 従来の表示
nc -zv host 80              # ポート疎通確認
traceroute example.com      # 経路表示
tracepath example.com
tcpdump -i eth0 port 80     # パケットキャプチャ
```

`ss` は `netstat` の後継です。`-t` TCP、`-u` UDP、`-l` 待ち受け、`-n` 名前解決しない、`-p` プロセス表示を意味します。

障害切り分けの基本順序は次のとおりです。

1. インターフェースが UP か（`ip link`）。
2. IP アドレスとルートがあるか（`ip addr`、`ip route`）。
3. 同一ネットワーク内に疎通するか（`ping`）。
4. デフォルトゲートウェイに疎通するか。
5. DNS が解決できるか（`dig`、`getent hosts`）。
6. 目的のポートが開いているか（`nc`、`ss`）。

7.4 クライアント側の DNS 設定

`/etc/resolv.conf` に DNS サーバーを指定します。

```
nameserver 192.168.1.1
nameserver 8.8.8.8
search example.com
options timeout:2 attempts:3
```

項目	意味
`nameserver`	問い合わせ先 DNS サーバー
`search`	短縮名に補うドメイン
`domain`	ローカルドメイン名
`options timeout:n`	タイムアウト秒数
`options attempts:n`	再試行回数

名前解決の順序は `/etc/nsswitch.conf` で決まります。

```
hosts: files dns
```

この場合、まず `/etc/hosts`（files）を参照し、見つからなければ DNS を参照します。`getent hosts example.com` は、アプリケーションと同じ順序で名前解決を確認できます。

本章のまとめ

- IPv4 は32ビット、IPv6 は128ビット。CIDR 表記に慣れる。
- プライベートアドレスと主要ポート（22、25、53、80、123、443）は暗記する。
- TCP は信頼性、UDP は高速性、ICMP は診断。
- 設定は `ip`／`nmcli`、確認は `ss`／`dig`／`ping`／`traceroute`。
- 名前解決は `/etc/hosts`、`/etc/resolv.conf`、`/etc/nsswitch.conf` の順で理解する。

練習問題

練習問題 7.1　`192.168.1.0/24` で利用可能なホストアドレスの数はどれか。

(a) 128　(b) 254　(c) 256　(d) 512

練習問題 7.2　DNS の標準ポート番号はどれか。

(a) 22　(b) 25　(c) 53　(d) 123

練習問題 7.3　TCP の説明として正しいものはどれか。

(a) コネクションレスで再送を行わない　(b) コネクション型で順序保証がある　(c) 診断専用のプロトコルである　(d) ポート番号を使わない

練習問題 7.4　待ち受け中の TCP ポートとプロセスを表示するコマンドはどれか。

(a) `ss -tulnp`　(b) `ping -c 4`　(c) `dig +short`　(d) `ip route`

練習問題 7.5　`/etc/nsswitch.conf` の `hosts: files dns` の意味はどれか。

(a) DNS を先に参照する　(b) `/etc/hosts` を先に参照し、次に DNS を参照する　(c) DNS を使わない　(d) 常にブロードキャストで解決する

第8章 システム管理

この章で学ぶこと

- ユーザーとグループの追加・変更・削除
- /etc/passwd、/etc/shadow、/etc/group の構造
- パスワードエージングとアカウントのロック
- cron／at／anacron によるジョブスケジューリング
- ロケールと文字コードの設定

8.1 アカウント管理

ユーザーとグループの操作

```
sudo useradd -m -s /bin/bash -G developers alice
sudo passwd alice
sudo usermod -aG sudo alice        # グループを追加（-a を忘れない）
sudo usermod -s /bin/zsh alice
sudo userdel -r alice              # ホームディレクトリも削除
sudo groupadd developers
sudo groupdel developers
sudo groupmod -n devs developers   # グループ名変更
```

useradd はホームディレクトリを自動作成しません（ディストリビューションの設定による）。-m を付けると /etc/skel の内容をコピーしてホームを作成します。-s はログインシェル、-G は補助グループ、-g は主グループです。

usermod -aG の -a は「追加」を意味します。-a を忘れると、既存の補助グループが置き換わるため注意してください。

アカウントデータベース

/etc/passwd の各フィールドは次のとおりです。

```
alice:x:1000:1000:Alice:/home/alice:/bin/bash
```

フィールド	意味
1	ユーザー名
2	パスワード（x は shadow に格納されていることを示す）
3	UID
4	GID

フィールド	意味
5	GECOS（コメント・氏名など）
6	ホームディレクトリ
7	ログインシェル

/etc/shadow にはハッシュ化されたパスワードとエージング情報が入ります。

```
alice:$6$...:19500:0:99999:7:::
```

フィールド	意味
1	ユーザー名
2	ハッシュ化パスワード（! や * はログイン不可）
3	最終パスワード変更日（1970-01-01 からの日数）
4	変更可能になるまでの日数（0は即時）
5	パスワードの有効期限（日数）
6	期限切れ前の警告日数
7	期限切れ後の猶予日数
8	アカウント失効日
9	予約

/etc/group にはグループ名、GID、所属ユーザーが入ります。

```
developers:x:1001:alice,bob
```

確認コマンド

```
id alice
getent passwd alice
getent group developers
groups alice
passwd -S alice          # パスワード状態
chage -l alice           # パスワードエージング情報
```

パスワードエージング

```
sudo chage -M 90 -W 7 -I 14 alice
sudo chage -E 2026-12-31 alice
sudo chage -d 0 alice      # 次回ログイン時に変更を強制
```

- `-M`: パスワードの最大有効日数
- `-m`: 変更可能になるまでの最小日数
- `-W`: 期限切れ前の警告日数
- `-I`: 期限切れ後の猶予日数
- `-E`: アカウントの失効日
- `-d`: 最終変更日

スケルトンディレクトリ

`/etc/skel/` は、新規ユーザーのホームディレクトリ作成時にコピーされる雛形です。`.bashrc`、`.profile` などを置いておくと、全ユーザーに同じ初期設定を配れます。

```
ls -la /etc/skel/
sudo useradd -m -k /etc/skel alice
```

8.2 ジョブスケジューリング

cron

cron は、決まった時刻にコマンドを実行する仕組みです。

```
分 時 日 月 曜日 コマンド
*  *  *  *  *    command
```

フィールド	範囲
分	0〜59
時	0〜23
日	1〜31
月	1〜12
曜日	0〜7（0と7が日曜）

```
crontab -e        # 自分の crontab を編集
crontab -l        # 一覧表示
crontab -r        # 削除
```

```
# 毎日 3:00 にバックアップ
0 3 * * * /usr/local/bin/backup.sh

# 5分ごと
*/5 * * * * /usr/bin/check.sh

# 平日 9:00〜17:00 の毎時0分
0 9-17 * * 1-5 /usr/bin/report.sh
```

システム全体の設定は `/etc/crontab` と `/etc/cron.d/` に置きます。`/etc/crontab` はコマンドの前に**実行ユーザー**を指定する点が、ユーザーの crontab と異なります。

```
0 3 * * * root /usr/local/bin/backup.sh
```

`/etc/cron.daily/`、`/etc/cron.hourly/`、`/etc/cron.weekly/`、`/etc/cron.monthly/` に置いたスクリプトは、それぞれの周期で実行されます。

cron のアクセス制御

```
/etc/cron.allow
/etc/cron.deny
```

`cron.allow` が存在する場合、そこに記載されたユーザーだけが cron を使えます。`cron.allow` がない場合は `cron.deny` に記載されていない全ユーザーが使えます。`at` も同様に `/etc/at.allow` と `/etc/at.deny` で制御します。

anacron

anacron は、常時起動していないマシンでも「1日1回」「1週間に1回」といった周期でジョブを実行する仕組みです。設定は `/etc/anacrontab` に記述します。

```
# period delay job-identifier command
1        5     daily-backup    /usr/local/bin/daily.sh
7        10    weekly-clean    /usr/local/bin/clean.sh
```

at

`at` は、指定した時刻に1回だけコマンドを実行します。

```
echo "/usr/local/bin/task.sh" | at 23:00
at -f task.sh 23:00
atq                # 予約一覧
atrm 3             # 予約番号3を削除
```

8.3 ローカライゼーションと国際化

国際化（i18n）は、ソフトウェアが複数の言語・地域に対応できるようにする設計、**ローカライゼーション（l10n）**は、特定の言語・地域向けに調整することです。

ロケール

```
locale                      # 現在のロケール設定
locale -a                   # 利用可能なロケール一覧
localectl status            # システムのロケールとキーボード
sudo localectl set-locale LANG=ja_JP.UTF-8
```

環境変数は次の優先順位で使われます。

- `LC_ALL`: すべてのロケール設定を上書きする（最優先）。
- `LC_*`: カテゴリ別の設定（`LC_TIME`、`LC_COLLATE`、`LC_MESSAGES` など）。
- `LANG`: 既定のロケール。

スクリプトで言語に依存しない出力を得たい場合は `LANG=C` や `LC_ALL=C` を使います。ソート順やメッセージが英語・C ロケールで固定され、処理が安定します。

文字コード

```
iconv -f ISO-2022-JP -t UTF-8 input.txt > output.txt
file -i text.txt
```

主な文字コードには ASCII、ISO-8859、UTF-8、ISO-2022-JP などがあります。現在の Linux では UTF-8 が標準的です。文字化けが起きる場合は、`file` や `iconv` で文字コードを確認・変換します。

本章のまとめ

- `useradd -m` でホームを作成し、`usermod -aG` でグループを追加する。
- `/etc/passwd` はユーザー情報、`/etc/shadow` はパスワードとエージング情報。
- `chage` でパスワードの有効期限・警告・失効を設定する。
- cron は5つの時刻フィールド。`/etc/crontab` はユーザー欄がある。
- `cron.allow` がある場合は記載ユーザーのみ許可。
- ロケールは `LANG`、`LC_*`、`LC_ALL` の優先順位を理解する。

練習問題

練習問題 8.1　新規ユーザー alice を作成し、ホームディレクトリも作成するコマンドはどれか。

(a) `useradd alice`　(b) `useradd -m alice`　(c) `usermod -m alice`　(d) `groupadd alice`

練習問題 8.2　alice を既存の補助グループ developers に追加するコマンドはどれか。

(a) `usermod -G developers alice`　(b) `usermod -aG developers alice`　(c) `groupmod -a alice developers`　(d) `useradd -G developers alice`

練習問題 8.3　毎日午前3時に `/usr/local/bin/backup.sh` を実行する crontab の記述はどれか。

(a) `0 3 * * * /usr/local/bin/backup.sh`　(b) `3 0 * * * /usr/local/bin/backup.sh`　(c) `* 3 * * * /usr/local/bin/backup.sh`　(d) `0 3 * * 0 /usr/local/bin/backup.sh`

練習問題 8.4　`/etc/cron.allow` が存在する場合の動作として正しいものはどれか。

(a) 全ユーザーが cron を使える　(b) 記載されたユーザーだけが cron を使える　(c) 記載されたユーザーは cron を使えない　(d) cron が停止する

練習問題 8.5　すべてのロケール設定を一時的に C ロケールにする環境変数はどれか。

(a) `LANG=C`　(b) `LC_ALL=C`　(c) `LC_TIME=C`　(d) `TZ=C`

第9章 重要なシステムサービス

この章で学ぶこと

- システム時刻の設定と NTP／chrony による同期
- rsyslog によるログ出力と中央ログサーバー
- systemd journal の検索・管理
- logrotate によるログローテーション
- MTA の基本とメールエイリアス

9.1 システム時刻の管理

日付とタイムゾーン

```
date                                # 現在の日時
sudo date -s "2026-04-01 09:00:00" # 手動設定（NTP 利用時は通常不要）
timedatectl                         # 時刻・タイムゾーン・NTP 状態
sudo timedatectl set-timezone Asia/Tokyo
sudo timedatectl set-ntp true
timedatectl list-timezones | grep Tokyo
```

タイムゾーンの定義は `/usr/share/zoneinfo/` にあります。`/etc/localtime` は使用するタイムゾーンファイルへのシンボリックリンク（またはコピー）です。

```
ls -l /etc/localtime
cat /etc/timezone        # Debian 系
```

ハードウェアクロック

```
sudo hwclock --show
sudo hwclock --systohc     # システム時刻をハードウェアクロックへ
sudo hwclock --hctosys     # ハードウェアクロックをシステム時刻へ
```

NTP と chrony

NTP はネットワーク経由で時刻を同期するプロトコルです。現在は `chrony` が広く使われています。

```
chronyc sources -v
chronyc tracking
sudo systemctl status chronyd
```

`/etc/chrony.conf`（または `/etc/chrony/chrony.conf`）に同期先を設定します。

```
pool 2.pool.ntp.org iburst
```

従来の ntpd では /etc/ntp.conf を使い、ntpq -p で同期状態を確認します。ntpdate は一時的な時刻合わせに使われますが、現在は非推奨の場面もあります。

9.2 システムのログ

rsyslog

rsyslog は、ファシリティと優先度に基づいてログを振り分けます。

ファシリティ	主な用途
auth / authpriv	認証関連
cron	cron
daemon	デーモン
kern	カーネル
mail	メール
user	ユーザープロセス
local0〜local7	独自用途

優先度	意味
emerg	システム使用不能
alert	即時対応が必要
crit	重大
err	エラー
warning	警告
notice	通知
info	情報
debug	デバッグ

/etc/rsyslog.conf の書式は「ファシリティ.優先度　アクション」です。

```
authpriv.*      /var/log/secure
cron.*          /var/log/cron
*.info;mail.none;authpriv.none;cron.none  /var/log/messages
*.* @@logserver.example.com:514
```

@ は UDP、@@ は TCP で転送します。中央のログサーバーとして受け取る場合は、モジュールの読み込みと受信設定を行います。

```
module(load="imudp")
input(type="imudp" port="514")
```

設定変更後は `systemctl restart rsyslog` で反映します。`logger` コマンドでテストログを送れます。

```
logger -p local0.info "test message"
```

systemd journal

```
journalctl                    # 全ログ
journalctl -b                 # 今回の起動分
journalctl -u sshd            # ユニットで絞り込み
journalctl -f                 # リアルタイム表示
journalctl --since "2026-04-01 00:00" --until "2026-04-01 12:00"
journalctl -p err             # err 以上
journalctl -n 50              # 最新50行
journalctl --disk-usage
sudo journalctl --vacuum-time=7d
```

既定では journal は `/run/log/journal/` に保存され、再起動で消える場合があります。永続化するには `/var/log/journal/` を作成するか、`/etc/systemd/journald.conf` の `Storage=persistent` を設定します。

```
[Journal]
Storage=persistent
SystemMaxUse=1G
```

logrotate

logrotate は、ログのローテーション・圧縮・削除を自動化します。設定は `/etc/logrotate.conf` と `/etc/logrotate.d/` に置きます。

```
/var/log/myapp/*.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 app app
    sharedscripts
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
```

ディレクティブ	意味
`daily` / `weekly` / `monthly`	ローテーション周期
`rotate n`	n世代保持
`compress`	gzip で圧縮
`missingok`	ファイルがなくてもエラーにしない
`notifempty`	空ファイルはローテーションしない
`create`	新しいログファイルを作成
`postrotate`	ローテーション後の処理

9.3 メール配送エージェント（MTA）

MTA（Mail Transfer Agent）は、メールの配送を担うソフトウェアです。代表的なものに Postfix、Exim、Sendmail があります。LinuC レベル1では、詳細な設定ではなく**エイリアスと基本的な配送設定**が問われます。

エイリアス

`/etc/aliases` にメールの転送先を定義します。

```
root:   admin@example.com
staff:  alice, bob
support: "|/usr/local/bin/ticket.sh"
```

編集後は `newaliases` を実行してデータベースを更新します。

```
sudo newaliases
```

ローカル配送

ユーザーごとの転送は `~/.forward` に記述します。

```
alice@example.com
```

`mail` コマンドでメールを送信し、`mailq` でキューの状態を確認できます。

```
mail -s "Test" alice@example.com < message.txt
mailq
```

MTA のログは `/var/log/maillog`（RHEL 系）や `/var/log/mail.log`（Debian 系）に出力されます。

本章のまとめ

- 時刻は `timedatectl`、`date`、`hwclock` で管理し、NTP は chrony が主流。
- タイムゾーンは `/usr/share/zoneinfo/` と `/etc/localtime`。
- rsyslog は「ファシリティ.優先度 アクション」で振り分ける。
- journal は `journalctl` で検索。永続化は `/var/log/journal/`。
- logrotate は `daily`、`rotate`、`compress` などを組み合わせる。
- メールは `/etc/aliases` と `newaliases`、`~/.forward` を覚える。

練習問題

練習問題 9.1　タイムゾーンを Asia/Tokyo に設定するコマンドはどれか。

(a) `timedatectl set-timezone Asia/Tokyo`　(b) `date Asia/Tokyo`　(c) `hwclock Asia/Tokyo`　(d) `ntpdate Asia/Tokyo`

練習問題 9.2　rsyslog で認証関連のログを `/var/log/secure` に出力する設定はどれか。

(a) `authpriv.* /var/log/secure`　(b) `secure.* authpriv`　(c) `*.* /var/log/secure`　(d) `authpriv.* @@/var/log/secure`

練習問題 9.3　今回の起動に関する journal を表示するコマンドはどれか。

(a) `journalctl -b`　(b) `journalctl -f`　(c) `journalctl -u`　(d) `journalctl --vacuum-time=7d`

練習問題 9.4　logrotate で7世代分のログを保持するディレクティブはどれか。

(a) `rotate 7`　(b) `daily 7`　(c) `keep 7`　(d) `compress 7`

練習問題 9.5　`/etc/aliases` を編集した後に実行するコマンドはどれか。

(a) `mailq`　(b) `newaliases`　(c) `postfix stop`　(d) `aliases -u`

第10章 セキュリティ

この章で学ぶこと

- セキュリティ監査と、SUID／SGID ファイルの検索
- ログイン状況・プロセス・メモリ使用量の確認と制限
- sudo／su の違いと sudoers の設定
- ホストのセキュリティ設定と不要サービスの停止
- SSH 鍵、GPG、暗号化アルゴリズムの基礎
- クラウドセキュリティの基礎

10.1 セキュリティ管理業務

監査

```
find / -perm -4000 -type f 2>/dev/null     # SUID ファイル
find / -perm -2000 -type f 2>/dev/null     # SGID ファイル
find / -nouser -o -nogroup 2>/dev/null     # 所有者不明ファイル
```

SUID が設定されたファイルは、一般ユーザーが所有者（しばしば root）の権限で実行できます。不要な SUID ファイルは攻撃対象になるため、定期的に監査します。

ログイン状況と履歴

```
who             # 現在ログイン中のユーザー
w               # ログイン中のユーザーと実行中のコマンド
last            # 最近のログイン履歴
lastb           # 失敗したログイン（要 root）
lastlog         # 各ユーザーの最終ログイン
```

プロセス・ポート・ファイルの確認

```
ss -tulnp                  # 待ち受けポートとプロセス
netstat -tulnp             # 従来のコマンド
nmap -sT localhost         # ポートスキャン
lsof -i :80                # 80番ポートを使うプロセス
fuser -n tcp 80            # 80番ポートを使う PID
```

nmap は自ホストの監査に使えますが、許可なく他者のホストをスキャンすることは避けてください。

リソース制限

```
ulimit -a          # すべての制限
ulimit -u          # ユーザーあたりのプロセス数
ulimit -n          # オープンファイル数
ulimit -c          # コアダンプサイズ
```

/etc/security/limits.conf でユーザーやグループごとの制限を設定できます。

sudo と su

```
su -               # root に切り替え（root のパスワードが必要）
su - alice         # alice に切り替え
sudo command       # sudoers の設定に従って権限を借りて実行
sudo -l            # 自分に許可された sudo コマンド
sudo -u www-data command
```

/etc/sudoers は必ず visudo で編集します。構文エラーを防ぎ、同時編集の競合を避けるためです。

```
%wheel ALL=(ALL) ALL
alice ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart httpd
```

%wheel はグループ、ALL=(ALL) は「すべてのホストで、すべてのユーザーとして」実行可能という意味です。NOPASSWD: を付けるとパスワード入力が不要になります。

自動ログアウト

```
export TMOUT=600
```

TMOUT は、指定秒数操作がない場合にシェルを終了します。/etc/profile などで設定できます。

10.2 ホストのセキュリティ設定

シャドウパスワード

パスワードハッシュを /etc/shadow に分離し、一般ユーザーから読めないようにする仕組みです。/etc/passwd のパスワード欄が x になっていれば shadow が使われています。

```
ls -l /etc/shadow     # 通常は root:shadow 600 または 640
```

ログイン禁止

/etc/nologin を作成すると、root 以外のユーザーのログインを拒否できます。ファイルの内容がログイン時に表示されます。

```
sudo sh -c 'echo "メンテナンス中です" > /etc/nologin'
sudo rm /etc/nologin
```

不要なサービスの停止

```
systemctl list-unit-files --type=service
sudo systemctl stop telnet.socket
sudo systemctl disable telnet.socket
sudo systemctl disable --now httpd
```

`disable` は自動起動を無効にし、`--now` を付けると同時に停止します。従来の `chkconfig` や `service` コマンドも試験範囲です。

```
chkconfig --list
chkconfig httpd off
service httpd stop
```

`inetd`／`xinetd` は、スーパーサーバーとして必要時にサービスを起動する仕組みです。設定は `/etc/inetd.conf` や `/etc/xinetd.conf`、`/etc/xinetd.d/` に置かれます。現在は systemd に置き換えられていますが、概念として出題されます。

ファイアウォール

```
sudo iptables -L -n -v
sudo firewall-cmd --list-all
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --reload
```

`iptables` はパケットフィルタの伝統的なツールです。`firewalld` はゾーンとサービス単位で管理する高レベルなフロントエンドです。Ubuntu では `ufw` もよく使われます。

```
sudo ufw status
sudo ufw allow 22/tcp
sudo ufw enable
```

10.3 暗号化によるデータの保護

暗号化の基本

種類	特徴	用途
共通鍵暗号	同じ鍵で暗号化・復号	大量データの暗号化（AES など）
公開鍵暗号	公開鍵で暗号化、秘密鍵で復号	鍵交換、署名、SSH

種類	特徴	用途
ハッシュ関数	一方向の変換	完全性確認、パスワード保存

SSH 鍵

```
ssh-keygen -t ed25519 -C "alice@example.com"
ssh-keygen -t rsa -b 4096
ssh-add ~/.ssh/id_ed25519
eval "$(ssh-agent -s)"
ssh-copy-id user@server
```

アルゴリズム	特徴
RSA	古くから使われる。鍵長は3072ビット以上が望ましい
ECDSA	楕円曲線暗号。鍵長が短い
Ed25519	高速で安全。現在の推奨

SSH ポート転送

```
ssh -L 8080:localhost:80 user@server   # ローカル転送
ssh -R 8080:localhost:80 user@server   # リモート転送
ssh -D 1080 user@server                # 動的転送（SOCKS プロキシ）
```

- `-L`: 手元のポートをリモート側のサービスへ転送する。
- `-R`: リモート側のポートを手元のサービスへ転送する。
- `-D`: SOCKS プロキシとして動的転送する。

GnuPG

```
gpg --gen-key
gpg --list-keys
gpg --export -a alice > alice.asc
gpg --import alice.asc
gpg --encrypt -r alice@example.com file.txt
gpg --decrypt file.txt.gpg
gpg --sign file.txt
gpg --verify file.txt.sig
```

GnuPG の鍵や設定は `~/.gnupg/` に保存されます。`gpg-agent` は秘密鍵のパスフレーズをキャッシュし、署名や復号を補助します。

10.4 クラウドセキュリティの基礎

パブリッククラウドでは、**責任共有モデル**が基本です。

- クラウド事業者: 物理設備、ハイパーバイザー、基盤ネットワークなど。
- 利用者: OS の設定、アプリケーション、アカウント管理、データ、セキュリティグループなど。

クラウド特有の注意点

項目	内容
管理コンソール	インターネット経由でアクセス。多要素認証（MFA）を必須にする
ファイアウォール	セキュリティグループ／ネットワーク ACL で制御
ルーティング	VPC／サブネット／ルートテーブルで設計
リージョン	データの所在、法規制、遅延を考慮して選択
揮発性ストレージ	インスタンス停止でデータが消える場合がある。永続ストレージと分ける
メンテナンス	事業者側のメンテナンスや再起動の影響を理解する

クラウドでは、オンプレミスのように物理的なネットワーク機器を直接管理しません。管理コンソールの認証、IAM、セキュリティグループの設定が重要になります。

本章のまとめ

- SUID／SGID ファイルは `find -perm` で監査する。
- ログイン状況は `who`／`w`／`last`、ポートは `ss`／`nmap` で確認する。
- `sudo` は sudoers に従って権限を借り、`su` はユーザーを切り替える。
- 不要サービスは `systemctl disable --now` で停止する。
- SSH は Ed25519 が推奨。`-L`／`-R`／`-D` の違いを覚える。
- クラウドは責任共有モデルと MFA、セキュリティグループが重要。

練習問題

練習問題 10.1　SUID が設定されたファイルを検索するコマンドはどれか。

(a) `find / -perm -4000 -type f`　(b) `find / -perm -2000 -type f`　(c) `find / -perm -1000 -type f`　(d) `find / -type d -perm 777`

練習問題 10.2　現在ログイン中のユーザーと実行中のコマンドを表示するコマンドはどれか。

(a) `who`　(b) `w`　(c) `last`　(d) `lastb`

練習問題 10.3　`/etc/sudoers` を安全に編集するコマンドはどれか。

(a) `vi /etc/sudoers`　(b) `visudo`　(c) `sudoedit /etc/passwd`　(d) `chmod 777 /etc/sudoers`

練習問題 10.4　手元の8080番ポートを、リモートサーバーから見たlocalhost:80へ転送する SSH オプションはどれか。

(a) `ssh -L 8080:localhost:80 user@server`　(b) `ssh -R 8080:localhost:80 user@server`　(c) `ssh -D 8080 user@server`　(d) `ssh -X user@server`

練習問題 10.5　クラウドの責任共有モデルで、利用者が責任を持つものはどれか。

(a) 物理サーバーの保守　(b) ハイパーバイザーの脆弱性対応　(c) OS の設定とアカウント管理　(d) データセンターの電源

第11章 オープンソースの文化

この章で学ぶこと

- オープンソースの定義と特徴
- 主要なライセンス（GPL、LGPL、AGPL、MPL、Apache、MIT、BSD）の違い
- コピーレフトと寛容型ライセンス
- オープンソースコミュニティとエコシステムへの参加方法

11.1 オープンソースの概念

オープンソースは、単に「ソースコードが公開されている」というだけではありません。OSI（Open Source Initiative）は、**オープンソースの定義**として次の10項目を定めています。

1. 自由な再頒布（Free Redistribution）
2. ソースコードの公開（Source Code）
3. 派生作品の作成と頒布（Derived Works）
4. 原著作者のソースコードの完全性（Integrity of The Author's Source Code）
5. 個人・グループに対する差別の禁止
6. 利用分野に対する差別の禁止
7. ライセンスの頒布
8. ライセンスが特定の製品に依存しないこと
9. ライセンスが他のソフトウェアを制限しないこと
10. ライセンスが技術中立であること

オープンソースの特徴として、次の点も押さえます。

- ソースコードが公開され、自由に利用・改変・再配布できる。
- 著作権は維持される（パブリックドメインを除く）。
- 原則として無保証である。
- コミュニティによって継続的に開発されることが多い。
- 多数のライセンスが存在し、条件が異なる。

11.2 主要なライセンス

ライセンス	種類	主な特徴
GPLv2／GPLv3	強いコピーレフト	派生作品も GPL で公開する必要がある
AGPL	強いコピーレフト	ネットワーク経由の利用でもソース公開を要求
LGPL	弱いコピーレフト	ライブラリの利用時は、利用側コードの公開を求めない
MPL 2.0	ファイル単位のコピーレフト	改変したファイルのみソース公開
Apache License 2.0	寛容型	特許許諾を含む。商用利用可

ライセンス	種類	主な特徴
MIT	寛容型	非常に簡潔。著作権表示と免責のみ
BSD（2条／3条）	寛容型	再配布時に著作権表示を保持。3条は名称使用を制限
パブリックドメイン	著作権なし	自由に利用できるが、保証もない

コピーレフト

コピーレフトは、ソフトウェアの自由を維持するために、派生作品にも同じライセンス条件を課す考え方です。GPL はその代表で、改変・再配布時にソースコードの提供が求められます。

- **強いコピーレフト**: GPL、AGPL。作品全体に影響する。
- **弱いコピーレフト**: LGPL。ライブラリとリンクする部分に限定。
- **寛容型（permissive）**: MIT、BSD、Apache。改変・再配布の条件が緩い。

ライセンス選択の注意

ライセンスは、ソフトウェアの利用方法やビジネスモデルに影響します。たとえば、GPL のコードを組み込んだ製品を配布する場合、ソースコードの提供義務が生じることがあります。Apache 2.0 は特許許諾条項を含むため、特許リスクを考慮する企業に選ばれます。

11.3 コミュニティとエコシステム

オープンソースプロジェクトは、次のような仕組みで運営されます。

要素	役割
メーリングリスト／フォーラム	議論・質問・告知
課題管理システム（Issue Tracker）	バグ報告・機能要望の管理
バージョン管理（Git など）	ソースコードの管理
プルリクエスト／コードレビュー	変更提案と品質確認
メンテナ	変更の採否、リリース管理
財団	法務・資金・ガバナンスの支援（Linux Foundation など）

参加方法は、大きく分けて次の3つです。

1. **使う**: ソフトウェアを導入し、フィードバックする。
2. **紹介する**: ブログや勉強会で広める。
3. **貢献する**: バグ報告、ドキュメント改善、コード提供、翻訳など。

良いバグ報告には、再現手順、環境情報（OS・バージョン）、期待する動作と実際の動作、ログが含まれます。相手を尊重し、既存の報告を検索してから投稿するのがマナーです。

本章のまとめ

- オープンソースは「自由な再頒布」「ソースコード公開」「派生作品の許容」など10項目で定義される。
- GPL は強いコピーレフト、LGPL は弱いコピーレフト、MIT／BSD／Apache は寛容型。
- AGPL はネットワーク利用でもソース公開を要求する。
- コミュニティ参加は「使う」「紹介する」「貢献する」の3段階。
- バグ報告は再現手順と環境情報を具体的に書く。

練習問題

練習問題 11.1　オープンソースの定義に含まれるものはどれか。

(a) ソースコードを非公開にできる　(b) 自由な再頒布を認める　(c) 特定の製品でのみ利用を許可する　(d) 個人の利用を禁止できる

練習問題 11.2　GPL の特徴として正しいものはどれか。

(a) 派生作品のソース公開を求めない　(b) 派生作品も GPL で公開する必要がある　(c) 商用利用を禁止する　(d) 特許許諾を必ず含む

練習問題 11.3　Apache License 2.0 の特徴として正しいものはどれか。

(a) 強いコピーレフトである　(b) 特許許諾条項を含む寛容型ライセンスである　(c) ソースコードの公開を禁止する　(d) ネットワーク利用時に必ずソース公開を求める

練習問題 11.4　オープンソースプロジェクトへの貢献方法として適切でないものはどれか。

(a) バグを再現手順付きで報告する　(b) ドキュメントの誤りを修正する　(c) 既存の報告を無視して同じ内容を大量投稿する　(d) 翻訳を改善する

第12章 総まとめと合格戦略

この章で学ぶこと

- 101試験・102試験の出題範囲と重要度の整理
- 頻出テーマの最終確認
- CBT 本番の時間配分と解き方
- 学習チェックリスト

12.1 出題範囲の全体像

試験	主題	内容	重要度の目安
101	1.01	インストール、仮想マシン・コンテナ	4
101	1.02	ファイル・ディレクトリ操作	3
101	1.03	GNU と Unix のコマンド	4
101	1.04	リポジトリとパッケージ管理	3
101	1.05	ハードウェア、ディスク、ファイルシステム	4
102	1.06	シェルおよびスクリプト	6
102	1.07	ネットワークの基礎	4
102	1.08	システム管理	5
102	1.09	重要なシステムサービス	5
102	1.10	セキュリティ	3
102	1.11	オープンソースの文化	2

重要度は公式出題範囲のトピックごとの重みです。特に 1.06（シェルスクリプト）は重要度が高く、配点も多くなります。1.08 と 1.09 も出題が多いため、cron、ログ、アカウント管理は確実に押さえましょう。

12.2 頻出テーマの最終確認

101試験

- インストール時の UEFI／GPT／ESP の関係
- `shutdown`、`reboot`、`halt`、`poweroff` の違い
- systemd のターゲットと `systemctl set-default`／`isolate`
- `ps`、`top`、`pgrep`、`kill`、`jobs`、`fg`、`bg`
- パーミッションの数値計算、SUID／SGID／スティッキービット、`umask`
- ハードリンクとシンボリックリンクの違い
- `find` の条件指定、FHS の主要ディレクトリ

- リダイレクト（`>`、`>>`、`2>&1`）、パイプ、`xargs`、`tee`
- 正規表現と `grep`、`sed` の置換
- `vi` のモード切替と保存・終了
- `apt`／`dpkg`、`yum`／`rpm` の対応関係
- パーティション、`/etc/fstab`、UUID、LVM の PV／VG／LV
- `mkfs`、`mount`、`swapon`、`blkid`

102試験

- ログインシェルと `~/.bash_profile`／`~/.bashrc` の読み込み
- `source`、`alias`、`PATH`、`&&`／`||`
- シェルスクリプトの引数（`$1`、`$#`、`$@`、`$?`）、`if`、`case`、`for`、`while`
- シバン、実行権、`set -euo pipefail`、`bash -x`
- IPv4／IPv6、CIDR、プライベートアドレス、主要ポート
- `ip addr`、`ip route`、`ss`、`dig`、`ping`、`traceroute`
- `/etc/resolv.conf`、`/etc/nsswitch.conf`、`/etc/hosts`
- `useradd`、`usermod`、`userdel`、`chage`、`/etc/passwd`、`/etc/shadow`
- crontab の書式、`/etc/cron.d`、`cron.allow`、`at`
- ロケール（`LANG`、`LC_ALL`）、`iconv`
- `timedatectl`、chrony、`journalctl`、`logrotate`、`/etc/aliases`
- `sudo`／`su`、`visudo`、`TMOUT`、`ss`／`nmap`／`lsof`
- SSH 鍵、`ssh-keygen`、`-L`／`-R`／`-D`、GPG
- クラウドの責任共有モデル、MFA、セキュリティグループ
- オープンソースライセンスの種類と特徴

12.3 本番の解き方

1. **最初に全問をざっと確認する**。時間のかかる問題を把握する。
2. **1問あたり90秒を目安**に進める。分からなければフラグを付けて次へ。
3. **選択肢は消去法**で絞る。似たコマンド名は「どの系統か」「何を操作するか」で判断する。
4. **コマンド入力問題**は、スペルとオプションを慎重に確認する。余計な空白や引用符に注意。
5. **見直しでは、フラグを付けた問題とマーク切替を優先**する。
6. 最後まで空欄を残さない。分からなくても最も妥当な選択肢を選ぶ。

12.4 学習チェックリスト

- [ ] 仮想マシンを1台構築し、SSH で接続できた
- [ ] ユーザーを追加し、グループ・パスワード・有効期限を設定できた
- [ ] パーティションを切り、ファイルシステムを作成してマウントできた
- [ ] `/etc/fstab` に UUID で登録し、再起動後もマウントされた
- [ ] `find` と `grep` を組み合わせて目的のファイルを探せた
- [ ] パイプとリダイレクトを使ってログを加工できた
- [ ] 引数・条件分岐・ループを含む Bash スクリプトを書けた

- [ ] cron で定期実行を設定し、`journalctl` で確認できた
- [ ] `ip`／`ss`／`dig` でネットワークを調査できた
- [ ] SSH の鍵認証とポート転送を設定できた
- [ ] `sudoers` を `visudo` で編集できた
- [ ] 主要なオープンソースライセンスの違いを説明できた

12.5 総仕上げ問題

練習問題 12.1　`/etc/fstab` の第1フィールドに UUID を指定する利点はどれか。

(a) デバイス名の変化に強い　(b) マウントが不要になる　(c) パーミッションが自動設定される　(d) ファイルシステムが不要になる

練習問題 12.2　Bash スクリプトで、未定義変数の参照をエラーにする設定はどれか。

(a) `set -e`　(b) `set -u`　(c) `set -x`　(d) `set -n`

練習問題 12.3　`/etc/shadow` に格納される情報はどれか。

(a) ユーザー名とホームディレクトリのみ　(b) ハッシュ化パスワードとエージング情報　(c) グループの一覧のみ　(d) DNS サーバーの設定

練習問題 12.4　`journalctl -u sshd` の意味はどれか。

(a) sshd ユニットのログを表示する　(b) sshd を再起動する　(c) sshd の設定を表示する　(d) sshd を停止する

練習問題 12.5　`ip route add default via 192.168.1.1` の意味はどれか。

(a) IP アドレスを追加する　(b) デフォルトゲートウェイを設定する　(c) インターフェースを起動する　(d) DNS を設定する

練習問題 12.6　「弱いコピーレフト」に分類されるライセンスはどれか。

(a) GPL　(b) LGPL　(c) MIT　(d) Apache 2.0

練習問題 12.7　`sudo` の説明として正しいものはどれか。

(a) root のパスワードを常に要求する　(b) sudoers の設定に従って権限を借りてコマンドを実行する　(c) すべてのユーザーに無制限の権限を与える　(d) ログインシェルを変更する

練習問題 12.8　クラウドで多要素認証（MFA）を導入する主な目的はどれか。

(a) 通信速度を上げる　(b) パスワード漏えい時の不正ログインリスクを下げる　(c) ストレージ容量を増やす　(d) OS のアップデートを自動化する

付録A 練習問題の解答

第1章

解答 1.1 （b） UEFI は ESP（EFI System Partition）からブートローダを読み込みます。ESP は FAT32 で、通常 `/boot/efi` にマウントします。

解答 1.2 （b） `shutdown -h +10` は10分後に停止します。`-r` は再起動、`-h` は停止です。

解答 1.3 （b） コンテナはホスト OS のカーネルを共有し、名前空間と cgroups で隔離します。ゲスト OS ごとにカーネルを持つのは仮想マシンです。

解答 1.4 （b） `set-default` はデフォルトのブートターゲットを変更します。`isolate` は一時的な切り替えです。

解答 1.5 （a） `kill -9` は SIGKILL を送り、プロセスを強制終了します。`-15` は SIGTERM です。

第2章

解答 2.1 （a） ファイルの基準 666 から 027 を引くと 640 になります（グループは読み取りのみ、その他は権限なし）。

解答 2.2 （b） `/tmp` は誰でも書き込めますが、スティッキービットにより他ユーザーのファイルは削除できません。

解答 2.3 （b） ハードリンクは同じ inode を共有します。異なるファイルシステム間では作成できません。

解答 2.4 （a） `-type f` は通常ファイル、`-mtime -7` は7日以内に更新されたファイルを意味します。

解答 2.5 （c） システム設定ファイルは `/etc` に置かれます。

第3章

解答 3.1 （a） `2>&1` は標準エラー出力（fd 2）を標準出力（fd 1）へ合流させます。

解答 3.2 （a） `/etc/passwd` は `:` 区切りなので、`-d:` と `-f1` で1列目を取り出します。

解答 3.3 （b） `sed -i` はファイルを直接編集し、`g` は各行のすべての一致を置換します。

解答 3.4 （b） `-print0` と `xargs -0` を組み合わせると、空白を含むファイル名も安全に扱えます。

解答 3.5 （c） `:q!` は変更を破棄して終了します。`:wq` と `ZZ` は保存して終了です。

第4章

解答 4.1 （b） `apt update` はリポジトリのパッケージ情報を更新します。システムの更新は `apt upgrade` です。

解答 4.2 （b） `dpkg -S` は指定したファイルを含むインストール済みパッケージを検索します。

解答 4.3 （b） `yum provides` はファイルや機能を提供するパッケージを検索します。

解答 4.4 （b） `yumdownloader` はパッケージをダウンロードするだけでインストールしません。

解答 4.5 (b) `r` は削除済み、`c` は設定ファイルが残っている状態です。

第5章

解答 5.1 (a) `lsmod` は読み込まれているカーネルモジュールを一覧表示します。

解答 5.2 (b) ESP は通常 `/boot/efi` にマウントします。

解答 5.3 (b) PV を作成し、VG にまとめ、そこから LV を切り出します。

解答 5.4 (b) XFS は拡張できますが、縮小はサポートされていません。

解答 5.5 (c) 6番目は fsck の実行順序です。ルートは1、その他は2、不要なら0です。

第6章

解答 6.1 (b) `source`（または `.`）は、現在のシェルで設定ファイルを読み込みます。

解答 6.2 (b) `$1` は第1引数です。`$0` はスクリプト名、`$#` は引数の数、`$?` は終了ステータスです。

解答 6.3 (c) `$?` は直前のコマンドの終了ステータスです。

解答 6.4 (c) `"$@"` は各引数を個別の文字列として展開します。

解答 6.5 (c) `bash -n` は実行せずに構文だけをチェックします。

第7章

解答 7.1 (b) `/24` は256個のアドレスのうち、ネットワークアドレスとブロードキャストアドレスを除いた254個が利用可能です。

解答 7.2 (c) DNS は TCP と UDP の53番ポートを使います。

解答 7.3 (b) TCP はコネクション型で、再送制御と順序保証があります。

解答 7.4 (a) `ss -tulnp` は TCP/UDP の待ち受けポートとプロセスを表示します。

解答 7.5 (b) `files` は `/etc/hosts`、`dns` は DNS サーバーを意味し、記載順に参照します。

第8章

解答 8.1 (b) `useradd -m` はホームディレクトリを作成し、`/etc/skel` の内容をコピーします。

解答 8.2 (b) `-aG` は既存の補助グループを維持したまま追加します。

解答 8.3 (a)「分 時 日 月 曜日」の順で、`0 3 * * *` は毎日3時0分です。

解答 8.4 (b) `cron.allow` が存在する場合、記載されたユーザーだけが cron を利用できます。

解答 8.5 (b) `LC_ALL` はすべてのロケールカテゴリを上書きします。

第9章

解答 9.1 (a) `timedatectl set-timezone` でタイムゾーンを設定します。

解答 9.2 (a) `authpriv.*` は認証関連のすべての優先度を意味します。

解答 9.3 (a) `journalctl -b` は今回の起動に関するログを表示します。

解答 9.4 （a） `rotate 7` は7世代分を保持します。

解答 9.5 （b） `/etc/aliases` を変更したら `newaliases` でデータベースを更新します。

第10章

解答 10.1 （a） `-perm -4000` は SUID ビットが設定されたファイルを検索します。

解答 10.2 （b） `w` はログイン中のユーザーと実行中のコマンドを表示します。

解答 10.3 （b） `visudo` は構文チェックを行いながら sudoers を編集します。

解答 10.4 （a） `-L` はローカルポート転送です。手元の8080番をリモートのlocalhost:80へ転送します。

解答 10.5 （c） クラウド事業者は物理基盤を、利用者は OS 設定・アプリ・アカウント・データなどを担当します。

第11章

解答 11.1 （b） オープンソースの定義には自由な再頒布が含まれます。特定の製品や個人に限定することはできません。

解答 11.2 （b） GPL は強いコピーレフトで、派生作品も GPL で公開する必要があります。

解答 11.3 （b） Apache 2.0 は寛容型で、特許許諾条項を含みます。

解答 11.4 （c） 既存の報告を無視した重複投稿は、コミュニティの負担を増やします。まず検索し、必要な情報を添えて報告します。

第12章

解答 12.1 （a） UUID はデバイス名（`/dev/sdb1` など）が変わっても同じファイルシステムを識別できます。

解答 12.2 （b） `set -u` は未定義変数の参照をエラーにします。`-e` はコマンド失敗時の終了です。

解答 12.3 （b） `/etc/shadow` にはハッシュ化パスワードとエージング情報が格納されます。

解答 12.4 （a） `-u` は systemd のユニットで絞り込みます。

解答 12.5 （b） `default via` はデフォルトゲートウェイを指定します。

解答 12.6 （b） LGPL は弱いコピーレフトです。GPL は強いコピーレフト、MIT と Apache は寛容型です。

解答 12.7 （b） `sudo` は sudoers の設定に従い、許可された権限でコマンドを実行します。

解答 12.8 （b） MFA はパスワード以外の要素も要求し、認証情報の漏えいによる不正ログインのリスクを下げます。

付録B コマンド早見表

ファイル・ディレクトリ

コマンド	用途
`ls -la`	隠しファイルを含む詳細一覧
`cp -r` / `mv` / `rm -r`	コピー／移動／再帰削除
`mkdir -p` / `rmdir`	親ごと作成／空ディレクトリ削除
`chmod` / `chown` / `chgrp`	権限／所有者／グループ変更
`umask`	新規ファイルの権限マスク
`ln` / `ln -s`	ハードリンク／シンボリックリンク
`find` / `locate` / `which` / `whereis`	ファイル・コマンド検索
`tar` / `gzip` / `xz` / `dd`	アーカイブ・圧縮・コピー

テキスト処理

コマンド	用途
`cat` / `less` / `head` / `tail`	表示
`grep` / `egrep` / `fgrep`	検索
`sed`	置換・削除・抽出
`cut` / `sort` / `uniq` / `wc`	列・並べ替え・重複・集計
`tr` / `xargs` / `tee`	文字変換・引数変換・分岐出力

プロセス・システム

コマンド	用途
`ps aux` / `top` / `pstree`	プロセス表示
`pgrep` / `pkill` / `kill` / `killall`	プロセス検索・シグナル
`jobs` / `fg` / `bg` / `nohup`	ジョブ管理
`systemctl`	systemd の操作
`journalctl`	journal の表示
`crontab -e` / `at`	定期実行／一回実行
`timedatectl` / `chronyc`	時刻・NTP

ネットワーク

コマンド	用途
`ip addr` / `ip route` / `ip link`	アドレス・経路・リンク
`ss -tulnp` / `netstat`	ポート・接続
`ping` / `traceroute` / `tracepath`	疎通・経路
`dig` / `host` / `getent hosts`	DNS 問い合わせ
`nmcli`	NetworkManager 操作
`nc`	ポート疎通確認
`tcpdump`	パケットキャプチャ

パッケージ・ストレージ

コマンド	用途
`apt update` / `apt install` / `apt remove`	Debian 系パッケージ
`dpkg -i` / `dpkg -l` / `dpkg -S`	Debian パッケージ直接操作
`yum install` / `yum provides`	Red Hat 系パッケージ
`rpm -ivh` / `rpm -qa` / `rpm -qf`	RPM 直接操作
`lsblk` / `fdisk` / `gdisk` / `parted`	ディスク・パーティション
`mkfs` / `mkswap` / `swapon`	ファイルシステム・スワップ
`mount` / `umount` / `blkid`	マウント・UUID
`pvcreate` / `vgcreate` / `lvcreate`	LVM

セキュリティ

コマンド	用途
`ssh` / `ssh-keygen` / `ssh-copy-id`	SSH 接続・鍵
`ssh -L` / `-R` / `-D`	ポート転送
`sudo` / `su` / `visudo`	権限昇格・sudoers
`gpg`	暗号化・署名
`find -perm -4000`	SUID 監査
`ss` / `nmap` / `lsof` / `fuser`	ポート・プロセス調査
`last` / `who` / `w`	ログイン履歴

付録C 用語集

用語	説明
inode	ファイルのメタデータ（所有者、権限、サイズ、データ位置）を管理する番号
FHS	Filesystem Hierarchy Standard。Linux のディレクトリ構造の標準
systemd	PID 1 として動作する init システム。ユニットとターゲットで管理
ターゲット	systemd の動作状態。従来のランレベルに相当
ESP	EFI System Partition。UEFI ブートに必要な FAT32 パーティション
GPT	GUID Partition Table。大容量ディスク向けのパーティションテーブル
LVM	物理ボリュームをまとめて論理ボリュームを作る管理機能
スワップ	メモリ不足時にディスクへ退避する領域
UUID	ファイルシステムを一意に識別する値。`/etc/fstab` で利用
umask	新規ファイル・ディレクトリ作成時に剥奪する権限
SUID	実行時にファイル所有者の権限で動作するビット
SGID	実行時にグループ権限で動作、またはディレクトリのグループ継承
スティッキービット	ディレクトリ内のファイルを所有者以外が削除できないようにするビット
ハードリンク	同じ inode を指す別名。異なるファイルシステム間では不可
シンボリックリンク	リンク先のパスを保持する特殊ファイル
リポジトリ	パッケージを公開・配布する場所
依存関係	パッケージが動作するために必要な別のパッケージ
CIDR	IP アドレスのネットワーク部を `/24` のように表す記法
NAT	プライベート IP とグローバル IP を変換する技術
DNS	ドメイン名と IP アドレスを対応付ける仕組み
NTP	ネットワーク経由で時刻を同期するプロトコル
chrony	現在広く使われる NTP 実装
rsyslog	ファシリティと優先度でログを振り分けるデーモン
journal	systemd が管理するログ。`journalctl` で閲覧
logrotate	ログのローテーション・圧縮・削除を自動化する仕組み
MTA	Mail Transfer Agent。メール配送を担うソフトウェア
PAM	Pluggable Authentication Modules。認証方式をモジュール化する仕組み
sudoers	sudo の権限設定ファイル。`visudo` で編集

用語	説明
GPG	GnuPG。公開鍵暗号による暗号化・署名
コピーレフト	派生作品にも同じライセンス条件を課す考え方
寛容型ライセンス	MIT／BSD／Apache など、改変・再配布の条件が緩いライセンス

付録D 参考情報

公式情報

内容	URL
LinuC レベル1 試験概要	https://linuc.org/linuc1/
101試験 出題範囲	https://linuc.org/linuc1/range/101.html
102試験 出題範囲	https://linuc.org/linuc1/range/102.html
LinuC 公式サイト	https://linuc.org/
LPI-Japan	https://lpi.or.jp/

学習に役立つ情報

- LinuC 公式の「例題解説」: https://linuc.org/study/samples/
- Linux 標準教科書シリーズ: https://linuc.org/textbooks/linux/
- `man` ページと `--help` を習慣にする。
- 実際に仮想マシンを構築し、設定ファイルを変更して再起動で確認する。

免責

本書は LinuC レベル1 Version 10.0 の公式出題範囲を参考に、学習用に独自にまとめたものです。試験制度・出題範囲・料金などは変更される可能性があります。受験前には必ず公式サイトで最新情報を確認してください。コマンドの動作はディストリビューションやバージョンによって異なる場合があります。